SE 422 · Bölüm Özetleri

Introduction to Data Science · Tüm bölümlerin konu anlatımı

16 bölüm
7 Kasım 2026

İçindekiler

Bölüm 1 · Python Temelleri 6

Bölümün amacı 6

Konu anlatımı 6

Sık yapılan hatalar 12

Kısa tekrar soruları 13

Cevaplar 13

Bölüm 2 · Karar ve Döngü Yapıları 14

Bölümün amacı 14

Konu anlatımı 14

Sık yapılan hatalar 19

Kısa tekrar soruları 19

Cevaplar 20

Bölüm 3 · Fonksiyonlar 21

Bölümün amacı 21

Konu anlatımı 21

Sık yapılan hatalar 26

Kısa tekrar soruları 27

Cevaplar 27

Bölüm 4 · Dosyalar ve İstisnalar 28

Bölümün amacı 28

Konu anlatımı 28

Sık yapılan hatalar 32

Kısa tekrar soruları 33

Cevaplar 33

Bölüm 5 · Listeler ve Demetler 34

Bölümün amacı 34

Konu anlatımı 34

Sık yapılan hatalar 38

Kısa tekrar soruları . . . 39
Cevaplar . . . 39

Bölüm 6 · String'ler . . . 40
Bölümün amacı . . . 40
Konu anlatımı . . . 40
Sık yapılan hatalar . . . 43
Kısa tekrar soruları . . . 44
Cevaplar . . . 44

Bölüm 7 · Sözlükler ve Kümeler . . . 45
Bölümün amacı . . . 45
Konu anlatımı . . . 45
Sık yapılan hatalar . . . 49
Kısa tekrar soruları . . . 49
Cevaplar . . . 50

Bölüm 8 · Sınıflar ve Nesne Yönelimli Programlama . . . 51
Bölümün amacı . . . 51
Konu anlatımı . . . 51
Sık yapılan hatalar . . . 55
Kısa tekrar soruları . . . 55
Cevaplar . . . 56

Bölüm 9 · Kalıtım ve Çok Biçimlilik . . . 57
Bölümün amacı . . . 57
Konu anlatımı . . . 57
Sık yapılan hatalar . . . 60
Kısa tekrar soruları . . . 61
Cevaplar . . . 61

Bölüm 10 · Özyineleme . . . 62
Bölümün amacı . . . 62
Konu anlatımı . . . 62
Sık yapılan hatalar . . . 66

Kısa tekrar soruları 66
Cevaplar 66

Bölüm 11 · Veritabanı Programlama 68
Bölümün amacı 68
Konu anlatımı 68
Sık yapılan hatalar 74
Kısa tekrar soruları 74
Cevaplar 74

Bölüm 12 · Python Ortamları 76
Bölümün amacı 76
Konu anlatımı 76
Sık yapılan hatalar 79
Kısa tekrar soruları 79
Cevaplar 79

Bölüm 13 · NumPy Bölüm 1 81
Bölümün amacı 81
Konu anlatımı 81
Sık yapılan hatalar 87
Kısa tekrar soruları 87
Cevaplar 88

Bölüm 14 · NumPy Bölüm 2 89
Bölümün amacı 89
Konu anlatımı 89
Sık yapılan hatalar 94
Kısa tekrar soruları 95
Cevaplar 95

Bölüm 15 · Pandas 96
Bölümün amacı 96
Konu anlatımı 96
Sık yapılan hatalar 106

Kısa tekrar soruları 107
Cevaplar 107

Bölüm 16 · Ödev 1 – Kavrayış özeti 109

Bölümün amacı 109
Konu anlatımı 109
Sık yapılan hatalar 115
Kısa tekrar soruları 116
Cevaplar 117

Bölüm 1 · Python Temelleri

Bölümün amacı

Bu bölüm bir Python programının temel döngüsünü öğretir: **girdi al → işle → çıktı ver**. Sonunda şunları yapabilmelisin:

- Python yorumlayıcısını etkileşimli modda ve betik modunda kullanmak
- String, yorum ve değişkenlerle çalışmak; veri tiplerini (int, float, str) tanımak
- Klavyeden girdi almak ve sayıya dönüştürmek, matematik operatörleriyle hesap yapmak
- `print` ve f-string ile biçimli çıktı üretmek
- İsimli sabitler, `if` / `if-else` ile karar vermek; ilişkisel ve mantıksal operatörleri kullanmak

Konu anlatımı

Yorumlayıcı ve iki çalışma modu (Slayt 5–8)

Python kodunu okuyup çalıştıran programa **yorumlayıcı** (interpreter) denir. İki şekilde kullanılır:

- **Etkileşimli mod:** komutları tek tek yazarsın; `>>>` istemi yorumlayıcının komut beklediğini gösterir. Yazılanlar kaydedilmez, denemeler için idealdir.
- **Betik modu:** komutlar `.py` uzantılı bir dosyaya yazılır ve komut satırında `python dosyaadi.py` ile çalıştırılır.

IDLE, Python ile birlikte kurulan; yazma, çalıştırma ve test araçlarını bir araya getiren, etkileşimli modda çalışan ve yerleşik metin editörü olan bir geliştirme ortamıdır.

String'ler ve yorumlar (Slayt 9–10)

String, veri olarak kullanılan karakter dizisidir. Kodda doğrudan yazılan string'e **string literal** denir; tek ya da çift tırnak içine yazılır. Üç tırnak (`'''` veya `"""`) hem iki tırnak türünü içerebilir hem de birden çok satıra yayılabilir.

`#` ile başlayan **yorumlar** insan okuyucu içindir; yorumlayıcı onları yok sayar.

```python
print("Kate'in kitabı")  # çift tırnak içinde tek tırnak kullanılabilir
print('''Birinci satır
İkinci satır''')
```

ÇIKTI

```
Kate'in kitabı
Birinci satır
İkinci satır
```

Değişkenler ve atama (Slayt 11–15)

Değişken, bellekteki bir değeri temsil eden isimdir. Atama ifadesiyle oluşturulur: `degisken = ifade`. Değeri alan değişken her zaman **solda** olur (`29 = age` hatadır). Değişken adı tırnak içine yazılmaz: `print('age')` değişkeni değil, "age" kelimesini yazar.

İsimlendirme kuralları: anahtar kelime olamaz, boşluk içeremez, harf veya _ ile başlar, büyük/küçük harfe duyarlıdır (`sayi` ≠ `Sayi`). Ad, değişkenin amacını anlatmalıdır (`x` yerine `saatlik_ucret`).

`print` birden çok öğeyi virgülle alır ve aralarına boşluk koyar. Bir değişkene sonradan yeni değer atanabilir; artık hiçbir değişkenin göstermediği değerler **çöp toplama** (garbage collection) ile bellekten silinir.

PYTHON

```python
age = 29
print(age)
print('age')
fiyat = 10
print('Fiyat:', fiyat, 'TL')
fiyat = 12
print('Yeni fiyat:', fiyat)
```

ÇIKTI

```
29
age
Fiyat: 10 TL
Yeni fiyat: 12
```

Veri tipleri (Slayt 16–17)

Değerler tiplere ayrılır: tam sayı **int**, ondalıklı sayı **float**, metin **str**. Ondalık noktası olmayan sayısal literal int (`7`), olan float'tır (`7.0`). Python'da bir değişken önce sayıyı, sonra metni gösterebilir.

PYTHON

```python
x = 99
print(type(x))
x = 'Take me to your leader'
print(type(x))
```

PYTHON (DEVAM)

```
print(type(7.0))
```

ÇIKTI

```
<class 'int'>
<class 'str'>
<class 'float'>
```

Klavyeden girdi ve tip dönüştürme (Slayt 18–19)

`input(istem)` klavyeden okur ve **her zaman string döndürür**. Sayı gerekiyorsa `int()` veya `float()` ile dönüştürülür; metin geçerli bir sayı değilse istisna (exception) oluşur.

PYTHON

```
metin = '42'          # input() bir string döndürmüş gibi düşün
sayi = int(metin)
print(sayi + 8)
print(float('3.5') * 2)
```

ÇIKTI

```
50
7.0
```

PYTHON

```
print(int('kırk iki'))
```

ÇIKTI

```
ValueError: invalid literal for int() with base 10: 'kırk iki'
```

Matematik operatörleri ve öncelik (Slayt 20–23)

Operatör	Anlamı	Örnek
`+ - *`	toplama, çıkarma, çarpma	`2 * 3` → 6
`/`	ondalıklı bölme	`7 / 2` → 3.5
`//`	tam sayı bölmesi	`7 // 2` → 3
`%`	bölümden kalan	`17 % 5` → 2
`**`	üs	`2 ** 3` → 8

Öncelik (yüksekten düşüğe): parantez → `**` → `* / // %` → `+ -`; aynı öncelikte soldan sağa. `//` pozitif sonucu keser, negatif sonucu sıfırdan uzağa yuvarlar.

PYTHON
```
print(7 / 2, 7 // 2, -7 // 2)
print(2 + 3 * 4, (2 + 3) * 4)
print(2 ** 3, 17 % 5)
```

ÇIKTI
```
3.5 3 -4
14 20
8 2
```

`%` tek/çift bulmada ve zaman dönüştürmede işe yarar:

PYTHON
```
saniye = 3725
print(saniye // 3600, 'saat', saniye % 3600 // 60, 'dakika', saniye % 60,
↪ 'saniye')
print(14 % 2 == 0)
```

ÇIKTI
```
1 saat 2 dakika 5 saniye
True
```

Karışık tipler ve uzun satırlar (Slayt 24–26)

İki int → int, iki float → float, karışık → float. `int(3.9)` ondalık kısmı **keser** (yuvarlamaz). Uzun ifadeler `\` ile ya da parantez içinde satırlara bölünebilir.

PYTHON
```
print(5 + 2.0)
print(int(3.9))
toplam = (1 + 2 +
          3 + 4)
print(toplam)
```

ÇIKTI
```
7.0
3
10
```

String birleştirme ve print ayrıntıları (Slayt 27–32)

`+` iki string'i birleştirir. Yan yana yazılan string literal'ler **örtük olarak** birleşir. `print` sonuna yeni satır koyar; `end=` bunu, `sep=` ise öğeler arasındaki boşluğu değiştirir. `\n` yeni satır, `\t` sekme karakteridir.

PYTHON
```python
print('Hello ' + 'world')
print('one' 'two' 'three')
print('A', 'B', 'C', sep='-')
print('Bir', end=' ')
print('İki')
print('Satır1\nSatır2\tsekmeli')
```

ÇIKTI
```
Hello world
onetwothree
A-B-C
Bir İki
Satır1
Satır2    sekmeli
```

f-string ile biçimli çıktı (Slayt 33–40)

Başına `f` konan string'de `{ }` yer tutucuları değişken ya da ifade alır. İki noktadan sonra **biçim belirteci** yazılır: sıra `[hizalama][genişlik][,][.hassasiyet][tip]`.

PYTHON
```python
name = 'Johnny'
num = 12345.6789
print(f'Hello {name}. The value is {10 + 2}.')
print(f'{num:.2f} | {num:,.2f} | {num:.2e}')
print(f'{0.5:.0%} | {123456789:,d}')
print(f'[{num:>12,.2f}] [{num:^12.1f}] [{num:<12.0f}]')
```

ÇIKTI
```
Hello Johnny. The value is 12.
12345.68 | 12,345.68 | 1.23e+04
50% | 123,456,789
[   12,345.68] [  12345.7   ] [12346       ]
```

Sihirli sayılar ve isimli sabitler (Slayt 41–44)

Kodda açıklamasız geçen sayılar (**sihirli sayı**) okunmayı zorlaştırır, değiştirmeyi zahmetli ve hataya açık yapar. Çözüm: büyük harfle yazılan **isimli sabit**.

PYTHON
```python
INTEREST_RATE = 0.069
balance = 1000
amount = balance * INTEREST_RATE
print(f'Faiz: {amount:.2f}')
```

ÇIKTI

```
Faiz: 69.00
```

if ve if-else ile karar verme (Slayt 45–57)

Sıra yapısında ifadeler sırayla çalışır; **karar yapısında** bazıları yalnızca bir koşul doğruysa çalışır. `if koşul:` satırının altındaki **girintili blok** koşul `True` ise çalışır. `else:` bloğu ise koşul yanlışken çalışır; `if` ve `else` aynı hizada yazılır.

Koşullar **ilişkisel operatörlerle** kurulur: `>`, `<`, `>=`, `<=`, `==` (eşit mi?), `!=` (eşit değil mi?). `=` atama, `==` karşılaştırmadır.

PYTHON

```python
sales = 60000
bonus = 500.0 if sales > 50000 else 0
temperature = 35
if temperature < 40:
    print("A little cold, isn't it?")
else:
    print("Nice weather we're having.")
print(bonus, 5 == 5, 5 != 5, 4 <= 5)
```

ÇIKTI

```
A little cold, isn't it?
500.0 True False True
```

String karşılaştırma (Slayt 58–59)

String'ler `==`, `!=`, `<`, `>` ile karşılaştırılabilir. Karşılaştırma **büyük/küçük harfe duyarlıdır** ve karakter karakter, karakter kodlarına göre yapılır. Kısa kelime uzun kelimenin başıysa uzun olan büyüktür.

PYTHON

```python
print('Mary' > 'Mark', 'a' == 'A', 'Ali' < 'Alice')
print(ord('y'), ord('k'))
```

ÇIKTI

```
True False True
121 107
```

Mantıksal operatörler, kısa devre ve Boolean değişkenler (Slayt 60–71)

- `and`: iki taraf da doğruysa True · `or`: biri doğruysa True · `not`: değeri ters çevirir.
- **Kısa devre:** `or`'da sol taraf True ise, `and`'de sol taraf False ise sağ taraf hiç değerlendirilmez.

- Aralık içi: `x >= 10 and x <= 20` · aralık dışı: `x < 10 or x > 20`.
- `in` / `not in` bir değerin bir dizide (ör. string) olup olmadığını sınar.
- **Boolean değişken** True/False tutar ve genellikle **bayrak** olarak kullanılır.

PYTHON
```
un, pwd = 'arda', '12345'
print(un == 'arda' and pwd == '12345')
hour, temp = 14, 28
print(hour < 15 or temp >= 28, not (hour <= 14))
x = 25
print(x >= 10 and x <= 20, x < 10 or x > 20)
print('s' in 'sebastian', '123' not in 'Password:12345')
```

ÇIKTI
```
True
True False
False True
True False
```

Sık yapılan hatalar

- **Değeri atanmadan değişkeni kullanmak** → `NameError`. Ayrıca `age` ile `Age` farklı değişkenlerdir.
- **input'un sonucunu sayı sanmak:** `input()` string döndürür; `'5' + 1` hata verir, önce `int()` gerekir.
- **String'e sayı eklemek:** `'Yaş: ' + 20` → `TypeError`. `str(20)` ya da f-string kullan.

PYTHON
```
yas = 20
print('Yaşın: ' + yas)
```

ÇIKTI
```
TypeError: can only concatenate str (not "int") to str
```

- `if` **satırının sonundaki** `:` **işaretini unutmak** veya `=` **ile** `==`**'yi karıştırmak** → `SyntaxError`.
- `int(3.9)`**'un yuvarladığını sanmak:** keser, sonuç 3'tür.
- `and`**'in iki tarafını tam yazmamak:** `x >= 10 and <= 20` geçersizdir; `x >= 10 and x <= 20` yazılır.

Kısa tekrar soruları

1. `print(7 // 2, -7 // 2, 7 % 2)` ne yazdırır?
2. `input()` hangi tipte değer döndürür? Bir sayıyla toplamak için ne yapmalısın?
3. `f'{1234.5:,.1f}'` ifadesinin sonucu nedir?
4. Sihirli sayı nedir? Neden isimli sabit tercih edilir?
5. `'apple' < 'apricot'` neden True'dur?
6. `x = 5` iken `x > 3 or 1 / 0 > 1` hata verir mi? Neden?

Cevaplar

1. `3 -4 1` (tam sayı bölmesi negatifte sıfırdan uzağa yuvarlar).
2. Her zaman `str`. Önce `int()` veya `float()` ile dönüştürmelisin.
3. `1,234.5`
4. Kodda açıklamasız geçen sayıdır. İsimli sabit kodu kendini açıklar hale getirir, değeri tek yerden değiştirmeyi sağlar ve yazım hatasını önler.
5. Karşılaştırma karakter karakter yapılır: `ap` aynı, üçüncü karakterde `p` (112) < `r` (114).
6. Vermez. `or`'da sol taraf True olduğu için kısa devre olur, `1 / 0` hiç çalışmaz.

Bölüm 2 · Karar ve Döngü Yapıları

Bölümün amacı

Bu bölüm programın akışını kontrol etmeyi öğretir: **birden çok koşula göre karar vermek** ve **bir işi tekrarlatmak**. Sonunda şunları yapabilmelisin:

- İç içe karar yapıları ve `if-elif-else` yazmak
- Koşul kontrollü (`while`) ve sayı kontrollü (`for` + `range`) döngüler kurmak
- Birikimli toplam, artırılmış atama, sentinel ve girdi doğrulama kalıplarını kullanmak
- `break`, `continue`, `pass` ve iç içe döngülerle çalışmak

Konu anlatımı

Hatırlatma: if-else ile fazla mesai (Slayt 1–3)

40 saati aşan saatlerin 1,5 katı ücretle ödendiği bir maaş hesabı, iki yollu bir karardır:

PYTHON
```
hours = 45
pay_rate = 20
if hours > 40:
    gross = 40 * pay_rate + (hours - 40) * pay_rate * 1.5
else:
    gross = hours * pay_rate
print(gross)
```

ÇIKTI
```
950.0
```

İç içe karar yapıları (Slayt 4–9)

Bir karar yapısı başka bir karar yapısının içine yerleştirilebilir. Örnek: kredi için **hem** yılda en az 30.000$ kazanmak **hem de** en az 2 yıldır çalışıyor olmak gerekir. Önce ilk koşul, o doğruysa ikinci koşul kontrol edilir.

`else` cümlesinin **hangi `if`'e ait olduğunu girinti belirler**: `else`, eşleştiği `if` ile aynı hizada yazılır.

PYTHON
```python
salary = 35000
years_on_job = 1
if salary >= 30000:
    if years_on_job >= 2:
        print('You qualify for the loan.')
    else:
        print('You must have been employed for at least 2 years.')
else:
    print('You must earn at least $30,000 per year.')
```

ÇIKTI
```
You must have been employed for at least 2 years.
```

if-elif-else (Slayt 10–15)

İç içe if-else'in derinleşen girintisini önler. Koşullar **yukarıdan aşağı** denenir; **ilk doğru olan** bloğu çalışır, gerisi atlanır. Hiçbiri doğru değilse `else` çalışır. İstediğin kadar `elif` eklenebilir.

PYTHON
```python
for temp in [120, 36, -5]:
    if temp > 99:
        print(temp, 'water is at boiling temperature')
    elif temp >= 1 and temp <= 99:
        print(temp, 'water temperature is at normal level')
    else:
        print(temp, 'water is at freezing temperature')
```

ÇIKTI
```
120 water is at boiling temperature
36 water temperature is at normal level
-5 water is at freezing temperature
```

Tekrar yapıları ve while (Slayt 16–21)

Aynı kodu kopyalamak programı büyütür ve düzeltmeyi zorlaştırır. **Tekrar yapısı** (döngü) bunu çözer. İki tür vardır: **koşul kontrollü** ve **sayı kontrollü**.

`while koşul:` koşul doğru olduğu sürece gövdeyi tekrarlar. Gövdenin bir kez çalışmasına **iterasyon** denir. `while` **ön testli** bir döngüdür: koşul baştan yanlışsa gövde hiç çalışmaz.

PYTHON
```python
i = 0
while i < 5:
    print(i, end=' ')
```

PYTHON (DEVAM)

```
    i += 1
print()
print('döngüden sonra i =', i)
```

ÇIKTI

```
0 1 2 3 4
döngüden sonra i = 5
```

Döngünün içinde koşulu eninde sonunda yanlış yapan bir şey olmalıdır; yoksa **sonsuz döngü** oluşur (ör. `i += 1` unutulursa).

for döngüsü ve range (Slayt 22–35)

`for degisken in dizi:` dizideki **her öğe için bir kez** döner. Her iterasyonun başında sıradaki öğe **hedef değişkene** atanır. Liste, string, tuple gibi yinelenebilir nesneler üzerinde çalışır.

PYTHON

```
for s in 'Star':
    print(s, end=' ')
print()
for renk in ['kırmızı', 'yeşil']:
    print(renk)
```

ÇIKTI

```
S t a r
kırmızı
yeşil
```

`range` sayı dizileri üretir: `range(stop)` 0'dan başlar; `range(start, stop)`'ta **stop dahil değildir**; üçüncü argüman adımdır, negatif adım geriye sayar.

PYTHON

```
print(list(range(5)), list(range(2, 6)), list(range(0, 10, 3)))
for n in range(1, 4):
    print(n, n ** 2)
print(list(range(19, 11, -1)))
```

ÇIKTI

```
[0, 1, 2, 3, 4] [2, 3, 4, 5] [0, 3, 6, 9]
1 1
2 4
3 9
[19, 18, 17, 16, 15, 14, 13, 12]
```

Birikimli toplam ve artırılmış atama (Slayt 36–38)

Bir serinin toplamı için **döngü** ve 0 ile başlayan bir **biriktirici** (accumulator) gerekir.

PYTHON
```python
total = 0
for n in [5, 10, 15]:
    total += n
print(total)
```

ÇIKTI
```
30
```

Operatör	Örnek	Eşdeğeri
`+=`	`x += 5`	`x = x + 5`
`-=`	`y -= 2`	`y = y - 2`
`*=`	`z *= 10`	`z = z * 10`
`/=`, `//=`, `%=`, `**=`	`a /= b`	`a = a / b`

Sentinel ve girdi doğrulama (Slayt 39–41)

Sentinel, bir öğe dizisinin sonunu işaretleyen özel değerdir ve normal bir değerle karışmayacak kadar ayırt edici olmalıdır. **GIGO** (garbage in, garbage out): kötü girdi kötü çıktı üretir. **Girdi doğrulama**, girdiyi işlemeden önce incelemektir; genellikle girdi kötü olduğu sürece dönen bir `while` döngüsüyle yapılır.

PYTHON
```python
girdiler = [12, 7, -1, 99]   # -1 sentinel: listenin sonunu gösterir
toplam = 0
for x in girdiler:
    if x == -1:
        break
    toplam += x
print(toplam)

denemeler = ['-5', 'abc', '30']   # klavyeden sırayla gelen girdiler gibi
sira = 0
yas = int(denemeler[sira]) if denemeler[sira].isdigit() else -1
while yas < 0:
    print('Geçersiz girdi:', denemeler[sira])
    sira += 1
    yas = int(denemeler[sira]) if denemeler[sira].isdigit() else -1
print('Kabul edilen yaş:', yas)
```

ÇIKTI

```
19
Geçersiz girdi: -5
Geçersiz girdi: abc
Kabul edilen yaş: 30
```

break, continue ve pass (Slayt 42–50)

- `break` içinde bulunduğu döngüyü **tamamen bitirir**; program döngüden sonraki satırdan devam eder.
- `continue` o iterasyonun **geri kalanını atlar**, sıradaki iterasyona geçer.
- `pass` hiçbir şey yapmaz; söz dizimi bir ifade gerektirip kod istemediğinde kullanılır.

PYTHON

```python
for w in 'gamer':
    if w == 'm':
        break
    print(w, end=' ')
print('| The loop has ended.')
for i in range(10):
    if i % 2 != 0:
        continue
    print(i, end=' ')
print()
for i in range(3):
    pass
print('pass bir şey yapmadı, i =', i)
```

ÇIKTI

```
g a | The loop has ended.
0 2 4 6 8
pass bir şey yapmadı, i = 2
```

İç içe döngüler (Slayt 51–56)

İç döngü, dış döngünün **her** iterasyonunda tüm iterasyonlarını tamamlar (analog saatte yelkovan ve saniye kolu gibi). Toplam iterasyon = iç × dış.

PYTHON

```python
for i in range(2):
    for j in range(3):
        print(i, ':', j, end='   ')
print()
sayac = 0
for saat in range(24):
    for dakika in range(60):
```

PYTHON (DEVAM)

```
        sayac += 1
print(sayac)
```

ÇIKTI

```
0 : 0   0 : 1   0 : 2   1 : 0   1 : 1   1 : 2
1440
```

Sık yapılan hatalar

- `else`**'i yanlış** `if` **ile hizalamak:** iç içe yapılarda mantığı tamamen değiştirir.
- `elif` **yerine** `else koşul:` **yazmak** → `SyntaxError`; `else` koşul almaz.

PYTHON

```
t = 50
if t > 99:
    print('boiling')
else t > 0:
    print('normal')
```

ÇIKTI

```
SyntaxError: invalid syntax
```

- `while`**'da koşulu değiştirmeyi unutmak** → sonsuz döngü.
- `range`**'in bitişi dahil sanmak:** `range(1, 5)` 1, 2, 3, 4 üretir.
- **Tam sayının üzerinde dönmeye çalışmak:** `for i in 5:` → `TypeError`; `range(5)` gerekir.
- **Biriktiriciyi döngünün içinde sıfırlamak:** `total = 0` döngüden önce yazılmalı.

Kısa tekrar soruları

1 `if-elif-else` yapısında birden çok koşul doğruysa hangi blok çalışır?

2 `while` neden "ön testli" bir döngüdür?

3 `list(range(10, 0, -3))` nedir?

4 `break` ile `continue` arasındaki fark nedir?

5 Dış döngü 4, iç döngü 5 kez dönüyorsa iç gövde toplam kaç kez çalışır?

6 Sentinel değeri seçerken neye dikkat edilir?

Cevaplar

1. Yalnızca ilk doğru koşulun bloğu.
2. Koşulu her iterasyondan önce test eder; koşul baştan yanlışsa gövde hiç çalışmaz.
3. `[10, 7, 4, 1]`
4. `break` döngüyü tamamen bitirir; `continue` yalnızca o iterasyonun kalanını atlayıp sıradakine geçer.
5. $4 \times 5 = 20$ kez.
6. Normal veri değerleriyle karışmayacak kadar ayırt edici olmalıdır.

Bölüm 3 · Fonksiyonlar

Bölümün amacı

Bu bölüm programı küçük, yeniden kullanılabilir parçalara bölmeyi öğretir. Sonunda şunları yapabilmelisin:

- Fonksiyon tanımlamak (`def`), çağırmak ve bir `main` fonksiyonuyla programı düzenlemek
- Yerel değişken, kapsam, argüman ve parametre kavramlarını ayırt etmek
- Pozisyonel ve anahtar kelimeli argüman kullanmak; global değişkenlerin risklerini bilmek
- `return` ile değer, Boolean, birden çok değer ve `None` döndürmek
- `random` ve `math` modüllerini, kendi modüllerini ve `__name__ == '__main__'` kalıbını kullanmak

Konu anlatımı

Fonksiyon nedir? (Slayt 1–6)

Fonksiyon, belirli bir görevi yerine getiren ifadeler grubudur. Büyük bir işi küçük görevlere bölüp her birini bir fonksiyona vermeye **böl ve yönet** denir; böyle yazılmış programa **modüler program** denir. Faydaları: daha basit kod, kodun yeniden kullanımı, daha kolay test ve ekip çalışması.

İki tür fonksiyon vardır:

- **Void fonksiyon:** ifadelerini çalıştırır ve biter.
- **Değer döndüren fonksiyon:** çağıran ifadeye bir değer geri gönderir (`input`, `int`, `float` gibi).

Tanımlama ve çağırma (Slayt 7–13)

Fonksiyon adı değişken adı kurallarına uyar ve genellikle bir fiil içerir. Tanım iki parçadır: **başlık** (`def ad():`) ve **girintili blok** (gövde). Fonksiyon **çağrıldığında** yorumlayıcı gövdeye atlar, ifadeleri çalıştırır ve çağrıldığı yere geri döner.

Programın ana mantığını bir `main` fonksiyonuna yazıp diğer fonksiyonları ondan çağırmak yaygın bir düzendir. Henüz yazılmamış fonksiyonlar `pass` ile iskelet olarak bırakılabilir.

PYTHON

```python
def main():
    print('Başla')
    selamla()
    hazirla()
    print('Bitir')

def selamla():
    print('Merhaba!')

def hazirla():
    pass   # daha sonra yazılacak

main()
```

ÇIKTI

```
Başla
Merhaba!
Bitir
```

Bloktaki satırlar aynı sayıda boşlukla girintilenmeli; sekme (Tab) ve boşluk **karıştırılmamalıdır**.

Yerel değişkenler ve kapsam (Slayt 14–15)

Bir fonksiyonun içinde değer atanan değişken **yereldir**: yalnızca o fonksiyondaki ifadeler erişebilir. Bir değişkene erişilebilen program bölümüne **kapsam** (scope) denir. Farklı fonksiyonlarda aynı isimli yerel değişkenler birbirini etkilemez.

PYTHON

```python
def texas():
    birds = 5000
    print('texas:', birds)

def california():
    birds = 8000
    print('california:', birds)

texas()
california()
```

ÇIKTI

```
texas: 5000
california: 8000
```

Argüman ve parametre (Slayt 16–25)

Argüman, fonksiyona gönderilen veridir; **parametre** onu alan değişkendir. Birden çok argüman **sırayla** (pozisyonel) eşleşir. **Anahtar kelimeli argümanla** (`parametre=deger`) sıra önemsiz hale gelir; karışık kullanımda pozisyonel argümanlar önce gelir.

Parametreye fonksiyon içinde yeni değer atamak, çağıranın değişkenini **değiştirmez** (değerle aktarım, pass by value).

PYTHON
```python
def show_sum(num1, num2):
    print(num1 + num2)

def show_interest(principal, rate, periods):
    print(principal * rate * periods)

def change_me(arg):
    arg = 0
    print('içeride:', arg)

show_sum(12, 45)
show_interest(rate=0.01, periods=10, principal=10000.0)
value = 99
change_me(value)
print('dışarıda:', value)
```

ÇIKTI
```
57
1000.0
içeride: 0
dışarıda: 99
```

Global değişkenler ve sabitler (Slayt 26–28)

Tüm fonksiyonların dışında oluşturulan değişken **globaldir**; her yerden okunabilir. Bir fonksiyon global değişkene **atama** yapacaksa içinde `global ad` bildirimi gerekir.

Global değişkenler hata ayıklamayı ve programı anlamayı zorlaştırır, fonksiyonları onlara bağımlı kılar; bu yüzden kaçınılmalıdır. Değişmeyen **global sabitler** ise uygundur.

PYTHON
```python
my_value = 10

def show_value():
    print(my_value)
```

PYTHON (DEVAM)

```python
def change():
    global my_value
    my_value = 50

show_value()
change()
show_value()
```

ÇIKTI

```
10
50
```

Standart kütüphane ve random modülü (Slayt 29–38)

Standart kütüphane, Python ile gelen hazır fonksiyonlardır; bunları **kara kutu** gibi kullanırız. Bazıları yerleşiktir (`print`, `range`), diğerleri **modüllerdedir** ve `import modul` ile yüklenip `modul.fonksiyon()` nokta gösterimiyle çağrılır.

`random` modülü: `randint(a, b)` (iki uç dahil), `randrange(...)` (range gibi), `random()` (0.0–1.0 arası float), `uniform(a, b)` (aralıkta float). Bu sayılar **sözde rastgeledir**: aynı **tohum** (`seed`) aynı seriyi üretir.

PYTHON

```python
import random
random.seed(10)
print(random.randint(1, 100), random.randrange(0, 10, 2))
print(round(random.random(), 4), round(random.uniform(1, 5), 4))
random.seed(10)
print(random.randint(1, 100))   # aynı tohum → aynı sayı
```

ÇIKTI

```
74 0
0.4289 3.3124
74
```

return ile değer döndürmek (Slayt 39–45)

`return ifade`, ifadenin değerini çağıran yere gönderir. Dönen değer bir değişkene atanabilir ya da başka bir fonksiyona verilebilir.

- **Boolean fonksiyon** True/False döndürür; koşulları ve girdi doğrulamayı sadeleştirir.
- **Birden çok değer:** `return a, b`; çağıran tarafta her değer için bir değişken gerekir.
- `None` "değer yok" demektir; ör. sıfıra bölmede hata işareti olarak döndürülebilir.

PYTHON
```python
def toplam(num1, num2):
    return num1 + num2

def is_even(n):
    return n % 2 == 0

def divide(num1, num2):
    if num2 == 0:
        return None
    return num1 / num2

def get_name():
    return 'Ada', 'Lovelace'

print(toplam(12, 45), is_even(4), divide(10, 0), divide(10, 4))
first, last = get_name()
print(last, first)
```

ÇIKTI
```
57 True None 2.5
Lovelace Ada
```

math modülü (Slayt 46–48)

`import math` ile karekök (`sqrt`), yukarı/aşağı yuvarlama (`ceil`, `floor`), üstel ve logaritma (`exp`, `log`, `log10`), trigonometri (radyan), `hypot` ve `math.pi`, `math.e` sabitleri kullanılabilir.

PYTHON
```python
import math
print(math.sqrt(16), math.ceil(4.2), math.floor(4.8))
print(math.hypot(3, 4), round(math.degrees(math.pi), 1))
radius = 2
print(round(math.pi * radius ** 2, 2))
```

ÇIKTI
```
4.0 5 4
5.0 180.0
12.57
```

Kendi modüllerin ve `__name__` (Slayt 49–56)

Modülerleştirme, ilgili fonksiyonları `.py` dosyalarında gruplamaktır; kodu anlamayı, test etmeyi ve yeniden kullanmayı kolaylaştırır. Bir modül fonksiyon **tanımları** içerir, çağrıları değil. **Menü güdümlü** programlar kullanıcıya işlemler listesi gösterip seçime göre çalışır.

Bir dosya yüklendiğinde `__name__` değişkeni oluşur: dosya doğrudan çalıştırıldıysa `'__main__'`, import edildiyse modülün adıdır. Böylece `main()` yalnızca dosya ana program olduğunda çalışır:

PYTHON
```python
def main():
    print('Ana program çalıştı, __name__ =', __name__)

if __name__ == '__main__':
    main()
```

ÇIKTI
```
Ana program çalıştı, __name__ = __main__
```

Sık yapılan hatalar

- **Fonksiyonu tanımlanmadan önce çağırmak** → `NameError`.

PYTHON
```python
selamla()
def selamla():
    print('Merhaba')
```

ÇIKTI
```
NameError: name 'selamla' is not defined
```

- **Argüman sayısını tutturamamak** → `TypeError`.

PYTHON
```python
def topla(a, b):
    return a + b
print(topla(5))
```

ÇIKTI
```
TypeError: topla() missing 1 required positional argument: 'b'
```

- **Yerel değişkene fonksiyon dışından erişmeye çalışmak** → `NameError`.
- `global` **bildirimi olmadan global değişkene** `+=` **yapmak** → `UnboundLocalError`.
- `return` **yazmayı unutmak:** fonksiyon `None` döndürür; `print(f())` "None" yazar.
- `import math` **yapmadan** `math.pi` **kullanmak** → `NameError`.

Kısa tekrar soruları

1. Void fonksiyon ile değer döndüren fonksiyonun farkı nedir?
2. `def f(a, b, c)` için `f(1, c=3, b=2)` geçerli midir? Peki `f(c=3, 1, 2)`?
3. Fonksiyon içinde parametreye yeni değer atamak çağıranın değişkenini değiştirir mi?
4. Global değişkenlerden neden kaçınılır?
5. `random.seed(5)` iki kez çağrılıp her seferinde `randint(1, 10)` istenirse sonuçlar aynı mı olur?
6. `if __name__ == '__main__':` kalıbı ne işe yarar?

Cevaplar

1. Void fonksiyon yalnızca ifadelerini çalıştırıp biter; değer döndüren fonksiyon çağıran ifadeye `return` ile bir değer gönderir.
2. İlki geçerlidir (pozisyonel önce, sonra anahtar kelimeli). İkincisi `SyntaxError` verir: pozisyonel argüman anahtar kelimeli argümandan sonra gelemez.
3. Hayır; değerle aktarım tek yönlüdür.
4. Hata ayıklamayı ve programı anlamayı zorlaştırır, fonksiyonları başka programlara taşımayı zorlaştırır.
5. Evet; aynı tohum aynı sözde rastgele seriyi üretir.
6. Dosya import edildiğinde `main()`'in otomatik çalışmasını önler; yalnızca doğrudan çalıştırıldığında çalışır.

Bölüm 4 · Dosyalar ve İstisnalar

Bölümün amacı

Program kapandığında değişkenlerdeki veri kaybolur. Bu bölüm veriyi **dosyada kalıcı olarak saklamayı** ve çalışma sırasında oluşan **hataları (istisnaları) zarifçe yönetmeyi** öğretir. Sonunda şunları yapabilmelisin:

- Metin dosyasını `open` ile açıp okumak, yazmak, sonuna eklemek ve kapatmak
- Sayıları dosyaya yazarken ve okurken dönüştürmek
- Dosyayı döngüyle satır satır işlemek; kayıt ve alan kavramlarını kullanmak
- `try / except / else / finally` ile istisnaları yakalamak

Konu anlatımı

Neden dosya? (Slayt 1–7)

Veriyi dosyaya kaydetmeye **yazmak**, dosyadan almaya **okumak** denir. Dosya kullanmanın üç adımı vardır: **aç → işle → kapat**.

- **Metin dosyası** veriyi metin olarak, **ikili (binary) dosya** ise metne çevrilmemiş biçimde tutar.
- **Sıralı erişimde** veri baştan sona okunur; **doğrudan erişimde** istenen konuma atlanabilir.

Dosya nesnesi ve open (Slayt 8–11)

`dosya = open(dosya_adi, mod)` dosyayı açar ve bir **dosya nesnesi** döndürür. Program dosyayla bu nesne üzerinden çalışır.

Mod	Anlamı
`'r'`	okuma; dosya yoksa hata
`'w'`	yazma; dosya varsa **içeriği silinir**, yoksa oluşturulur
`'a'`	ekleme; yazılanlar mevcut içeriğin **sonuna** gelir

Yol verilmezse dosya programla aynı klasörde aranır. Windows yolları için başa `r` konur: `open(r'C:\Users\data.txt')`; böylece ters eğik çizgiler kaçış karakteri sayılmaz.

Yazmak ve okumak (Slayt 12–14)

`write(string)` dosyaya yazar, `close()` kapatır. `read()` tüm içeriği tek string olarak, `readline()` bir satırı (`'\n'` dahil) okur. Dosya nesnesi bir **okuma konumu** tutar; her `readline` sıradaki satırı verir. Satır sonundaki `'\n'`, `rstrip('\n')` ile temizlenir.

PYTHON
```
f = open('philosophers.txt', 'w')
f.write('John Locke\n')
f.write('David Hume\n')
f.close()

f = open('philosophers.txt', 'r')
line1 = f.readline()
line2 = f.readline()
f.close()
print(repr(line1))
print(line1.rstrip('\n'), '/', line2.rstrip('\n'))
print(repr(open('philosophers.txt').read()))
```

ÇIKTI
```
'John Locke\n'
John Locke / David Hume
'John Locke\nDavid Hume\n'
```

Ekleme ve sayısal veri (Slayt 15–16)

`'a'` modu eski içeriği korur. Dosyaya **yalnızca string** yazılabilir; sayılar önce `str()` ile çevrilir. Okunan sayılar da string'dir, `int()` / `float()` ile çevrilmelidir.

PYTHON
```
f = open('numbers.txt', 'w')
f.write(str(10) + '\n')
f.close()
f = open('numbers.txt', 'a')
f.write(str(20) + '\n')
f.close()
f = open('numbers.txt', 'r')
toplam = int(f.readline()) + int(f.readline())
f.close()
print(toplam)
```

ÇIKTI
```
30
```

Dosyayı döngüyle işlemek (Slayt 17–19)

Dosyada kaç öğe olduğu genellikle bilinmez, bu yüzden döngü kullanılır. `readline` dosya sonunda **boş string** döndürür; bu bir **sentinel** gibi kullanılabilir (`while line`

`!= ''`). Daha kısası, `for line in dosya:` dosya bitince kendiliğinden durur.

PYTHON
```
f = open('sales.txt', 'w')
for s in [1000.0, 2000.0, 3000.0]:
    f.write(str(s) + '\n')
f.close()

total = 0
f = open('sales.txt', 'r')
for line in f:
    total += float(line)
f.close()
print(total)
```

ÇIKTI
```
6000.0
```

Kayıtlar ve alanlar (Slayt 20–21)

Kayıt tek bir öğeyi tanımlayan veri kümesidir (ör. bir çalışan); **alan** kayıttaki tek veri parçasıdır (ad, kimlik no). Sıralı dosyada alanlar art arda yazılır, okurken bir kaydın tüm alanları sırayla okunur. Kayıtlarla ekleme, görüntüleme, arama, değiştirme ve silme yapılır.

PYTHON
```
f = open('employees.txt', 'w')
for ad, no in [('Ingrid', '4587'), ('Julia', '4588')]:
    f.write(ad + '\n')
    f.write(no + '\n')
f.close()

f = open('employees.txt', 'r')
name = f.readline()
while name != '':
    id_num = f.readline()
    print(name.rstrip('\n'), id_num.rstrip('\n'))
    name = f.readline()
f.close()
```

ÇIKTI
```
Ingrid 4587
Julia 4588
```

İstisnalar ve traceback (Slayt 22–23)

İstisna (exception), çalışma sırasında oluşan ve programı genellikle durduran hatadır. **Traceback** hatanın türünü, açıklamasını ve satırını gösterir. Bazı istisnalar dikkatli kodla önlenebilir (bölmeden önce sıfır kontrolü); bazıları önlenemez (sayı olmayan metni çevirmek, olmayan dosyayı açmak).

PYTHON
```
print(int('abc'))
```

ÇIKTI
```
ValueError: invalid literal for int() with base 10: 'abc'
```

PYTHON
```
f = open('olmayan_dosya.txt', 'r')
```

ÇIKTI
```
FileNotFoundError: [Errno 2] No such file or directory: 'olmayan_dosya.txt'
```

try / except (Slayt 24–27)

İstisna fırlatabilecek kod `try` bloğuna, **istisna işleyici** `except` bloğuna yazılır. `try` içinde istisna oluşursa:

- Türü `except`'te belirtilen türse o blok çalışır ve program `try/except`'ten sonra devam eder.
- Başka bir türse program traceback ile durur.

Birden çok tür için ayrı `except`'ler yazılır; tür belirtmeyen `except` her şeyi yakalar ve en sona yazılır. `except ValueError as err:` ile istisnanın mesajı alınabilir.

PYTHON
```
for metin in ['42', 'kırk', '0']:
    try:
        sonuc = 100 / int(metin)
        print(metin, '→', sonuc)
    except ValueError as err:
        print(metin, '→ sayı değil:', err)
    except ZeroDivisionError:
        print(metin, '→ sıfıra bölünemez')
print('Program devam ediyor')
```

ÇIKTI
```
42 → 2.380952380952381
kırk → sayı değil: invalid literal for int() with base 10: 'kırk'
0 → sıfıra bölünemez
Program devam ediyor
```

else, finally ve işlenmeyen istisnalar (Slayt 28–31)

- `else` bloğu, `try` **istisnasız** biterse çalışır.
- `finally` bloğu istisna olsa da olmasa da **her zaman** çalışır; temizlik içindir (ör. dosyayı kapatmak).

İstisna, uygun türde `except` yoksa ya da `try`'ın dışında oluşmuşsa **işlenmemiş** kalır ve program durur.

PYTHON
```python
for deger in ['10', 'on']:
    try:
        sayi = int(deger)
    except ValueError:
        print(deger, '→ sayı değil')
    else:
        print(deger, '→', sayi * 2)
    finally:
        print('kontrol bitti')
```

ÇIKTI
```
10 → 20
kontrol bitti
on → sayı değil
kontrol bitti
```

Ek bilgi (resmi dokümantasyon): Dosyayı `with open(ad) as f:` ile açmak, blok bitince (hata olsa bile) dosyayı otomatik kapatır; `finally: f.close()` yazmanın kısa yoludur. Kaynak: docs.python.org/3/tutorial/inputoutput.html (Reading and Writing Files).

PYTHON
```python
with open('notlar.txt', 'w') as f:
    f.write('ilk satır\n')
with open('notlar.txt') as f:
    print(f.read().strip(), '| kapalı mı?', f.closed)
```

ÇIKTI
```
ilk satır | kapalı mı? False
```

Sık yapılan hatalar

- `'w'` **ile açarak eski içeriği silmek:** eklemek için `'a'` kullan.
- **Dosyaya sayı yazmak:** `f.write(10)` → `TypeError`; `str(10)` yaz.

PYTHON

```
f = open('rapor.txt', 'w')
f.write(10)
```

ÇIKTI

```
TypeError: write() argument must be str, not int
```

- **Okunan satırı sayı sanmak:** `'10\n'` bir string'dir; `int()` / `float()` gerekir.
- **Yanlış istisna türünü yakalamak:** `except ValueError` bir `ZeroDivisionError`'ı yakalamaz.
- **Dosyayı kapatmayı unutmak:** yazılanlar diske tam aktarılmayabilir; `close()` ya da `with` kullan.

Kısa tekrar soruları

1. `'w'` ve `'a'` modlarının farkı nedir?
2. `readline()` dosyanın sonuna gelince ne döndürür? Bu nasıl kullanılır?
3. Dosyadan okunan `'3.5\n'` satırıyla hesap yapmak için ne gerekir?
4. Kayıt ile alan arasındaki fark nedir?
5. `else` ve `finally` blokları hangi durumlarda çalışır?
6. Önlenebilen ve önlenemeyen birer istisna örneği ver.

Cevaplar

1. `'w'` dosya varsa içeriğini siler; `'a'` içeriği koruyup sonuna ekler.
2. Boş string (`''`) döndürür; `while line != ''` döngüsünde sentinel olarak kullanılır.
3. `float(satir)`; `float` sondaki `'\n'`'yi tolere eder, gerekirse önce `rstrip('\n')`.
4. Kayıt tek bir öğeyi tanımlayan veri kümesidir (bir çalışan); alan bu kaydın tek bir parçasıdır (adı).
5. `else` yalnızca `try` istisnasız bittiğinde; `finally` her durumda.
6. Önlenebilir: sıfıra bölme (öncesinde kontrol edilir). Önlenemez: kullanıcının sayı yerine metin girmesi ya da dosyanın olmaması.

Bölüm 5 · Listeler ve Demetler

Bölümün amacı

Bu bölüm birden çok değeri tek bir değişkende tutmayı öğretir. Sonunda şunları yapabilmelisin:

- Liste oluşturmak, indekslemek, dilimlemek ve değiştirmek
- Liste metotlarını (`append`, `insert`, `sort`, `remove`, `index`, `reverse`) ve `del`, `min`, `max`, `len` kullanmak
- Listeyi doğru şekilde kopyalamak; liste üreteci (list comprehension) yazmak
- İki boyutlu listelerle çalışmak; tuple ile listenin farkını açıklamak
- matplotlib ile çizgi, çubuk ve pasta grafiği çizmenin temel adımlarını bilmek

Konu anlatımı

Diziler, liste ve tuple (Slayt 1–7)

Dizi (sequence), öğeleri art arda saklanan nesnedir. **Liste** değiştirilebilir (mutable), **tuple** değiştirilemez (immutable) bir dizidir.

Liste köşeli parantezle yazılır ve farklı tipte öğeler tutabilir. `list()` bazı nesneleri (ör. `range`) listeye çevirir; `*` operatörü listeyi tekrarlar.

PYTHON
```
even_numbers = [2, 4, 6, 8, 10]
info = ['Alicia', 27, 1550.87]
print(info)
print(list(range(5)), [0] * 5)
for n in even_numbers:
    print(n, end=' ')
```

ÇIKTI
```
['Alicia', 27, 1550.87]
[0, 1, 2, 3, 4] [0, 0, 0, 0, 0]
2 4 6 8 10
```

İndeksleme ve len (Slayt 8–9)

İlk elemanın indeksi **0**, son elemanın `len(liste) - 1`'dir. Negatif indeksler sondan sayar (`-1` son eleman). Geçersiz indeks `IndexError` fırlatır.

PYTHON
```python
my_list = [10, 20, 30, 40]
print(my_list[0], my_list[-1], len(my_list))
index = 0
while index < len(my_list):
    print(my_list[index], end=' ')
    index += 1
```

ÇIKTI
```
10 40 4
10 20 30 40
```

Değiştirme ve birleştirme (Slayt 10–11)

Listeler mutable'dır: `liste[i] = yeni` ile eleman değişir. `+` iki listeyi birleştirir, `+=` mevcut listeye ekler.

PYTHON
```python
numbers = [1, 2, 3]
numbers[1] = 99
list1 = [1, 2]
list1 += [3, 4]
print(numbers, list1, [1, 2] + [5])
```

ÇIKTI
```
[1, 99, 3] [1, 2, 3, 4] [1, 2, 5]
```

Dilimleme ve in (Slayt 12–13)

`liste[start:end]` start'tan end'e (**end dahil değil**) bir **kopya** liste döndürür. Başlangıç yazılmazsa 0, bitiş yazılmazsa uzunluk kullanılır; üçüncü sayı adımdır. `in` / `not in` üyeliği sınar.

PYTHON
```python
days = ['Pzt', 'Sal', 'Çar', 'Per', 'Cum']
print(days[1:3], days[:2], days[::2], days[-2:])
print('Cum' in days, 'Cmt' not in days)
```

ÇIKTI
```
['Sal', 'Çar'] ['Pzt', 'Sal'] ['Pzt', 'Çar', 'Cum'] ['Per', 'Cum']
True True
```

Liste metotları (Slayt 14–17)

Metot	Ne yapar?
`append(x)`	sona ekler

Metot	Ne yapar?
`insert(i, x)`	`i` konumuna ekler, sonrakiler kayar
`index(x)`	ilk geçtiği indeks; yoksa `ValueError`
`remove(x)`	ilk geçtiği yeri siler; yoksa `ValueError`
`sort()` / `reverse()`	artan sıralar / ters çevirir

Ayrıca `del liste[i]` belirli indeksi siler; `min` ve `max` en küçük/en büyük değeri verir.

PYTHON
```
names = ['Ali', 'Can']
names.append('Ece')
names.insert(0, 'Zeynep')
print(names)
names.sort()
print(names)
names.remove('Can')
del names[0]
print(names, names.index('Zeynep'), min([4, 1, 7]), max([4, 1, 7]))
```

ÇIKTI
```
['Zeynep', 'Ali', 'Can', 'Ece']
['Ali', 'Can', 'Ece', 'Zeynep']
['Ece', 'Zeynep'] 1 1 7
```

Kopyalamak ve listeleri işlemek (Slayt 18–21)

`list2 = list1` **kopya oluşturmaz**: iki isim aynı listeyi gösterir. Gerçek kopya için elemanları yeni listeye eklemek ya da `[] + list1` yazmak gerekir. Toplam için biriktiricili döngü ya da `sum`, ortalama için `toplam / len` kullanılır. Listeler fonksiyonlara verilebilir ve fonksiyonlardan döndürülebilir; dosyaya her eleman `+ '\n'` ile yazılıp `readlines` ile okunabilir.

PYTHON
```
list1 = [1, 2, 3, 4]
ayni = list1
kopya = [] + list1
list1[0] = 99
print(ayni, kopya)
scores = [2.5, 7.3, 6.5]
print(round(sum(scores) / len(scores), 2))
```

ÇIKTI
```
[99, 2, 3, 4] [1, 2, 3, 4]
5.43
```

Liste üreteçleri (Slayt 22–28)

`[sonuc for oge in liste]` var olan listenin elemanları üzerinden yeni liste kurar. Sonuç ifadesi hesap içerebilir; sona eklenen `if` yalnızca koşulu sağlayan elemanları alır.

PYTHON

```
list1 = [1, 2, 3, 4]
print([item ** 2 for item in list1])
print([len(s) for s in ['Winken', 'Blinken', 'Nod']])
print([item for item in [1, 12, 2, 20, 3, 15, 4] if item < 10])
```

ÇIKTI

```
[1, 4, 9, 16]
[6, 7, 3]
[1, 2, 3, 4]
```

İki boyutlu listeler (Slayt 29–31)

Elemanları başka listeler olan listedir; satır ve sütun olarak düşünülür. Elemana `liste[satir][sutun]` ile erişilir, genellikle iç içe döngüyle işlenir.

PYTHON

```
scores = [[70, 80, 90],
          [85, 95, 75]]
print(scores[1][0])
for row in scores:
    for value in row:
        print(value, end=' ')
    print()
```

ÇIKTI

```
85
70 80 90
85 95 75
```

Tuple'lar (Slayt 32–34)

Tuple, oluşturulduktan sonra **değiştirilemeyen** bir dizidir: `(1, 2, 3)`. İndeksleme, dilimleme, `len`, `min`, `max`, `in`, `+`, `*` desteklenir; `append`, `remove`, `insert`, `sort`, `reverse` yoktur. Listeden hızlıdır ve veriyi güvenle korur. `list()` ve `tuple()` birbirine çevirir.

PYTHON

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple[0], len(my_tuple), max(my_tuple), my_tuple[1:3])
print(list(my_tuple), tuple([7, 8]))
```

ÇIKTI

```
1 5 5 (2, 3)
[1, 2, 3, 4, 5] (7, 8)
```

matplotlib ile grafik (Slayt 35–57)

matplotlib standart kütüphanede **değildir**; `pip install matplotlib` ile kurulur ve `import matplotlib.pyplot as plt` ile kullanılır. (Bu uygulamanın ortamında matplotlib kurulu olmadığı için aşağıdaki kodlar çalıştırılmadı; yalnızca gösterim amaçlıdır.)

- **Çizgi grafiği:** `plt.plot(x, y)`; `title`, `xlabel`, `ylabel`, `xlim`/`ylim`, `xticks`/`yticks`, `marker='o'`, `grid(True)`, sonra `plt.show()`.
- **Çubuk grafiği:** `plt.bar(sol_kenarlar, yukseklikler)`; varsayılan genişlik 0.8, `color=('r', 'g', 'b')`.
- **Pasta grafiği:** `plt.pie(degerler, labels=etiketler)`; her dilim değerin toplama oranıdır.

PYTHON

```python
import matplotlib.pyplot as plt
x_coords = [0, 1, 2, 3, 4]
y_coords = [0, 3, 1, 5, 2]
plt.plot(x_coords, y_coords, marker='o')
plt.title('Sales by Year')
plt.xlabel('Year')
plt.ylabel('Sales')
plt.xticks([0, 1, 2, 3, 4], ['2016', '2017', '2018', '2019', '2020'])
plt.grid(True)
plt.show()
```

PYTHON

```python
import matplotlib.pyplot as plt
plt.bar([0, 10, 20], [100, 200, 300], 5, color=('r', 'g', 'b'))
plt.show()
plt.pie([100, 400, 300], labels=['1. çeyrek', '2. çeyrek', '3. çeyrek'])
plt.show()
```

Sık yapılan hatalar

- **Son indeks için `len(liste)` kullanmak** → `IndexError`; son indeks `len(liste) - 1`'dir.

PYTHON

```python
l = [3, 5, 7]
print(l[3])
```

ÇIKTI

```
IndexError: list index out of range
```

- `list2 = list1` **ile kopyaladığını sanmak:** biri değişince diğeri de değişir.
- **Listede olmayan öğeyi** `remove` **/** `index` **ile aramak →** `ValueError`.
- **Tuple'ı değiştirmeye çalışmak:** `t.append(4)` → `AttributeError`; `t[0] = 9` → `TypeError`.
- `sort()`**'un sonucunu atamak:** `l = l.sort()` listeyi `None` yapar; `sort` listeyi yerinde sıralar ve `None` döndürür.

Kısa tekrar soruları

1 `[10, 20, 30, 40, 50][1:4]` nedir? `[...][-2:]` nedir?

2 `a = [1, 2]`, `b = a`, `b.append(3)` sonrası `a` nedir?

3 `[x * 2 for x in range(5) if x % 2 == 1]` nedir?

4 `m = [[1, 2, 3], [4, 5, 6]]` için `m[1][2]` nedir?

5 Tuple'ın listeye göre iki avantajını say.

6 matplotlib neden ayrıca kurulmalıdır?

Cevaplar

1 `[20, 30, 40]` ve `[40, 50]`.

2 `[1, 2, 3]`; `b` ve `a` aynı listeyi gösterir.

3 `[2, 6]`

4 `6`

5 İşlenmesi daha hızlıdır; değiştirilemediği için veriyi güvenle korur (ayrıca bazı işlemler tuple gerektirir).

6 Standart kütüphanenin parçası değildir.

Bölüm 6 · String'ler

Bölümün amacı

Programların büyük kısmı metinle çalışır. Bu bölüm string'leri incelemeyi ve dönüştürmeyi öğretir. Sonunda şunları yapabilmelisin:

- String'in karakterlerine döngü ve indeksle erişmek; uzunluğunu bulmak
- Birleştirme ve tekrar operatörlerini kullanmak; string'lerin neden değiştirilemez olduğunu açıklamak
- Dilimleme, `in` ve test/değiştirme/arama metotlarıyla çalışmak
- `split` ile string'i token'lara ayırmak

Konu anlatımı

String bir dizidir (Slayt 1–7)

String'ler **dizidir**; bu yüzden listelerde öğrendiğin araçların çoğu string'lerde de çalışır. Karakterlere iki yolla erişilir:

- **for döngüsü:** tüm karakterler üzerinde dolaşmak için (ör. bir harfi saymak)
- **İndeks:** `s[i]`, 0'dan başlar; negatif indeks sondan sayar. Aralık dışı indeks `IndexError` verir, `len(s)` uzunluğu döndürür.

PYTHON

```
name = 'Juliet'
for ch in name:
    print(ch, end=' ')
print()
my_string = 'Roses are red'
print(my_string[6], my_string[-1], len(my_string))
sayac = 0
for ch in 'banana':
    if ch == 'a':
        sayac += 1
print(sayac)
```

ÇIKTI

```
J u l i e t
a d 13
3
```

Birleştirme ve değişmezlik (Slayt 8–10)

`+` ve `+=` string'leri birleştirir. String'ler **değiştirilemez** (immutable): birleştirme var olan string'i değiştirmez, **yeni bir string** oluşturup değişkene atar. `s[0] = 'X'` gibi bir atama istisna fırlatır.

PYTHON
```
name = 'Carmen'
name = name + ' Brown'
letters = 'abc'
letters += 'def'
print(name, '|', letters)
```

ÇIKTI
```
Carmen Brown | abcdef
```

PYTHON
```
s = 'python'
s[0] = 'P'
```

ÇIKTI
```
TypeError: 'str' object does not support item assignment
```

Dilimleme ve in (Slayt 11–12)

`s[start:end]` bir **alt string** döndürür (end dahil değil). Başlangıç yoksa 0, bitiş yoksa uzunluk; adım ve negatif indeks kullanılabilir. `in` / `not in` bir string'in diğerinde geçip geçmediğini sınar.

PYTHON
```
full_name = 'Patty Lynn Smith'
print(full_name[0:5], '|', full_name[6:], '|', full_name[-5:], '|',
↪ full_name[::-1])
print('Lynn' in full_name, 'Ann' not in full_name)
```

ÇIKTI
```
Patty | Lynn Smith | Smith | htimS nnyL yttaP
True True
```

Test metotları (Slayt 13–14)

`s.metot()` biçimindeki bu metotlar True/False döndürür:

Metot	True, eğer string…
`isalnum()`	yalnızca harf veya rakam içeriyorsa
`isalpha()`	yalnızca harf içeriyorsa
`isdigit()`	yalnızca rakam içeriyorsa

Metot	True, eğer string...
`islower()` / `isupper()`	tüm harfleri küçük / büyükse
`isspace()`	yalnızca boşluk karakterleri içeriyorsa

PYTHON
```
print('abc123'.isalnum(), 'abc'.isalpha(), '2024'.isdigit())
print('Ali'.isupper(), 'ali'.islower(), '  \n'.isspace())
```

ÇIKTI
```
True True True
False True True
```

Değiştirme metotları (Slayt 15–16)

Bu metotlar string'i değiştirmez, **değiştirilmiş bir kopya** döndürür: `lower()`, `upper()`, `strip()` / `lstrip()` / `rstrip()` (uçlardaki boşlukları ya da verilen karakteri siler). Karşılaştırma büyük/küçük harfe duyarlı olduğundan kullanıcı girdisi genellikle `strip().lower()` ile normalize edilir.

PYTHON
```
cevap = '  Evet  '
print(cevap.strip().lower() == 'evet')
print('$$100$$'.strip('$'), 'python'.upper(), repr('  ab  '.rstrip()))
```

ÇIKTI
```
True
100 PYTHON '  ab'
```

Arama ve değiştirme (Slayt 17–19)

`startswith` / `endswith` başlangıç ve bitişi sınar. `find(alt)` alt string'in ilk indeksini, bulamazsa **-1** döndürür. `replace(eski, yeni)` her geçişi değiştirilmiş bir kopya üretir.

PYTHON
```
filename = 'rapor.txt'
print(filename.endswith('.txt'), filename.startswith('rap'))
string = 'Four score and seven years ago'
print(string.find('seven'), string.find('ten'))
print(string.replace('years', 'days'))
```

ÇIKTI
```
True True
15 -1
Four score and seven days ago
```

Tekrar operatörü (Slayt 20)

`string * n` string'in n kopyasını birleştirir; string solda, sayı sağdadır.

PYTHON
```
print('Ha' * 3)
print('-' * 20)
```

ÇIKTI
```
HaHaHa
--------------------
```

split ve token'lar (Slayt 21–27)

`split()` string'i bir **listeye** böler; varsayılan ayırıcı boşluktur, başka bir **sınırlayıcı** (delimiter) argüman olarak verilebilir. Sınırlayıcıyla ayrılmış parçalara **token**, bunları ayırma işine **tokenizing** denir.

PYTHON
```
print('One two three four'.split())
print('1/2/3/4/5'.split('/'))
tokens = 'www.example.com'.split('.')
print(tokens)
sayilar = '17;92;81;12;46;5'.split(';')
print(sayilar, sum(int(t) for t in sayilar))
```

ÇIKTI
```
['One', 'two', 'three', 'four']
['1', '2', '3', '4', '5']
['www', 'example', 'com']
['17', '92', '81', '12', '46', '5'] 253
```

Sık yapılan hatalar

- **String'i yerinde değiştirmeye çalışmak:** `s[0] = 'P'` → `TypeError`; yeni string oluştur: `'P' + s[1:]`.
- **Metotların string'i değiştirdiğini sanmak:** `s.upper()` tek başına `s`'yi değiştirmez; `s = s.upper()` yazılmalı.

PYTHON
```
s = 'merhaba'
s.upper()
print(s)
s = s.upper()
print(s)
```

```
ÇIKTI
merhaba
MERHABA
```

- `find` **sonucunu yanlış yorumlamak:** bulunamayınca hata değil `-1` döner; `if s.find(x):` yanlıştır (indeks 0 da False sayılır), `if x in s:` kullan.
- **Var olmayan metot çağırmak:** string'lerin `reverse()` metodu yoktur → `AttributeError`; tersi için `s[::-1]`.
- **Token'ı sayıya çevirmeden toplamak:** `split` her zaman string listesi döndürür.

Kısa tekrar soruları

1 `'Python'[1:4]` ve `'Python'[-3:]` nedir?

2 `name = 'Ada'; name += ' Lovelace'` satırı var olan string'i mi değiştirir?

3 `'hello world'.find('o')` ve `'hello'.find('z')` ne döndürür?

4 `' Merhaba '.strip().upper()` nedir?

5 `'a,b,,c'.split(',')` nedir?

6 `'3' * 3` ile `3 * 3` farkı nedir?

Cevaplar

1 `'yth'` ve `'hon'`.

2 Hayır; string'ler değiştirilemez. Yeni bir `'Ada Lovelace'` string'i oluşur ve `name` onu gösterir.

3 `4` (ilk geçtiği indeks) ve `-1`.

4 `'MERHABA'`

5 `['a', 'b', '', 'c']` (iki ayraç arasındaki boş token korunur).

6 `'3' * 3` → `'333'` (tekrar), `3 * 3` → `9` (çarpma).

Bölüm 7 · Sözlükler ve Kümeler

Bölümün amacı

Bu bölüm veriye **konumla değil anahtarla** erişmeyi (sözlük), **benzersiz öğe koleksiyonlarıyla** çalışmayı (küme) ve nesneleri dosyaya **serileştirmeyi** (pickle) öğretir. Sonunda şunları yapabilmelisin:

- Sözlük oluşturmak; değer almak, eklemek, değiştirmek, silmek; `get`, `items`, `keys`, `values`, `pop`, `popitem`, `clear` kullanmak
- Sözlük ve küme üreteçleri yazmak
- Küme oluşturmak; birleşim, kesişim, fark, simetrik fark, alt/üst küme işlemlerini yapmak
- `pickle` ile nesneleri dosyaya kaydetmek ve geri yüklemek

Konu anlatımı

Sözlük nedir? (Slayt 1–3)

Sözlük, her elemanı bir **anahtar** ve bir **değerden** oluşan koleksiyondur; anahtardan değere bir **eşlemedir**. Anahtar değiştirilemez (immutable) bir nesne olmalıdır (string, sayı, tuple); değerler herhangi bir tipte olabilir. `{anahtar: deger, ...}` ile yazılır.

PYTHON
```
phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
print(phonebook)
print(phonebook['Katie'])
```

ÇIKTI
```
{'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
555-2222
```

Değer almak, eklemek, silmek (Slayt 4–6)

- `d[anahtar]` değeri getirir; anahtar yoksa **KeyError**. `in` / `not in` önce kontrol etmeyi sağlar.
- `d[anahtar] = deger` yeni çift ekler; anahtar varsa değerini **değiştirir**.
- `del d[anahtar]` çifti siler; anahtar yoksa KeyError.

PYTHON
```python
phonebook = {'Chris': '555-1111'}
phonebook['Joe'] = '555-0123'
phonebook['Chris'] = '555-4444'
print('Joe' in phonebook, phonebook)
del phonebook['Joe']
print(phonebook)
```

ÇIKTI
```
True {'Chris': '555-4444', 'Joe': '555-0123'}
{'Chris': '555-4444'}
```

len, karışık tipler, boş sözlük ve döngü (Slayt 7–8)

`len` çift sayısını verir. Boş sözlük `{}` veya `dict()` ile oluşturulur. `for anahtar in d:` **anahtarlar** üzerinde dolaşır.

PYTHON
```python
test_scores = {'Kayla': [88, 92, 100], 'Luis': [95, 74, 81]}
print(len(test_scores))
for name in test_scores:
    print(name, sum(test_scores[name]) / len(test_scores[name]))
bos = {}
bos[42] = 'cevap'
print(bos)
```

ÇIKTI
```
2
Kayla 93.33333333333333
Luis 83.33333333333333
{42: 'cevap'}
```

Sözlük metotları (Slayt 9–13)

Metot	Ne yapar?
`get(k, varsayilan)`	değeri getirir; anahtar yoksa varsayılanı döndürür, **KeyError fırlatmaz**
`items()`	(anahtar, değer) tuple'ları
`keys()` / `values()`	anahtarlar / değerler
`pop(k, varsayilan)`	değeri döndürür ve çifti siler
`popitem()`	**son eklenen** çifti döndürür ve siler
`clear()`	sözlüğü boşaltır

PYTHON
```python
phonebook = {'Chris': '555-1111', 'Katie': '555-2222'}
print(phonebook.get('Andy', 'Bulunamadı'))
for key, value in phonebook.items():
    print(key, value)
d = {'a': 1, 'b': 2, 'c': 3}
print(list(d.keys()), d.pop('a'), d.popitem(), list(d.values()))
```

ÇIKTI
```
Bulunamadı
Chris 555-1111
Katie 555-2222
['a', 'b', 'c'] 1 ('c', 3) [2]
```

Sözlük üreteci (Slayt 14–19)

`{anahtar: deger for oge in dizi}` bir diziden sözlük üretir; `if` ile seçim yapılabilir. `{k: v for k, v in d.items()}` sözlüğü kopyalar.

PYTHON
```python
numbers = [1, 2, 3, 4]
print({item: item ** 2 for item in numbers})
print({item: len(item) for item in ['Jeremy', 'Kate', 'Peg']})
pop = {'New York': 8398748, 'Phoenix': 1660272, 'Houston': 2325502}
print({k: v for k, v in pop.items() if v > 2000000})
```

ÇIKTI
```
{1: 1, 2: 4, 3: 9, 4: 16}
{'Jeremy': 6, 'Kate': 4, 'Peg': 3}
{'New York': 8398748, 'Houston': 2325502}
```

Kümeler (Slayt 20–24)

Küme, matematiksel küme gibi davranır: öğeler **benzersizdir**, küme **sırasızdır** (indekslenemez). `set()` boş küme, `set(liste)` listeden küme oluşturur; string verilirse her karakter bir öğe olur. `add` bir öğe, `update` birden çok öğe ekler. `remove` öğe yoksa **KeyError** fırlatır, `discard` fırlatmaz.

PYTHON
```python
s = set([1, 2, 2, 3, 3, 3])
print(s, len(s), sorted(set('aabbc')))
s.add(4)
s.update([5, 6])
s.discard(10)
s.remove(2)
print(s, 3 in s)
```

ÇIKTI

```
{1, 2, 3} 3 ['a', 'b', 'c']
{1, 3, 4, 5, 6} True
```

Küme işlemleri (Slayt 25–30)

İşlem	Metot	Operatör	
Birleşim	`union`	`	`	`
Kesişim	`intersection`	`&`	
Fark	`difference`	`-`	
Simetrik fark	`symmetric_difference`	`^`	
Alt küme / üst küme	`issubset` / `issuperset`	`<=` / `>=`	

PYTHON

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(sorted(a | b), sorted(a & b), sorted(a - b), sorted(a ^ b))
print({1, 2} <= a, a.issuperset({1, 2}), a.union(b) == a | b)
```

ÇIKTI

```
[1, 2, 3, 4, 5, 6] [3, 4] [1, 2] [1, 2, 5, 6]
True True True
```

Küme üreteci (Slayt 31–34)

Liste üreteci gibi yazılır ama süslü parantez kullanılır; sonuç bir kümedir (tekrarlar kendiliğinden düşer).

PYTHON

```
set1 = set([1, 20, 2, 40, 3, 50])
print(sorted({item for item in set1 if item < 10}))
print(sorted({len(w) for w in ['ab', 'cd', 'efg']}))
```

ÇIKTI

```
[1, 2, 3]
[2, 3]
```

Nesneleri serileştirme: pickle (Slayt 35–39)

Serileştirme (pickling), bir nesneyi dosyada saklanabilecek bayt akışına dönüştürmektir. Adımlar: `import pickle` → dosyayı **ikili yazma** (`'wb'`) için aç → `pickle.dump(nesne, dosya)` → kapat. Geri almak (**unpickling**) için `'rb'` ile açıp `pickle.load(dosya)` çağrılır. Aynı dosyaya birden çok nesne yazılıp sırayla okunabilir.

PYTHON
```python
import pickle
phonebook = {'Chris': '555-1111', 'Katie': '555-2222'}
with open('phonebook.dat', 'wb') as f:
    pickle.dump(phonebook, f)
    pickle.dump([1, 2, 3], f)
with open('phonebook.dat', 'rb') as f:
    print(pickle.load(f))
    print(pickle.load(f))
```

ÇIKTI
```
{'Chris': '555-1111', 'Katie': '555-2222'}
[1, 2, 3]
```

Sık yapılan hatalar

- **Olmayan anahtarı `[]` ile istemek** → `KeyError`; `get` ya da `in` kullan.

PYTHON
```python
d = {'elma': 3}
print(d['kiraz'])
```

ÇIKTI
```
KeyError: 'kiraz'
```

- **Değiştirilebilir nesneyi anahtar yapmak:** liste anahtar olamaz → `TypeError: unhashable type`.
- **Kümeyi indekslemek:** `s[0]` → `TypeError`; kümeler sırasızdır.
- **`{}`'nin boş küme olduğunu sanmak:** `{}` boş **sözlüktür**; boş küme `set()` ile oluşturulur.

PYTHON
```python
print(type({}), type(set()))
```

ÇIKTI
```
<class 'dict'> <class 'set'>
```

- **pickle dosyasını metin modunda açmak:** `'wb'` / `'rb'` gerekir.

Kısa tekrar soruları

1 `d = {'a': 1}` için `d.get('b', 0)` ve `d['b']` ne verir?

2 Bir sözlükte anahtar olarak hangi tipler kullanılabilir?

3 `{x: x % 3 for x in range(5)}` nedir?

4 `set('mississippi')` kaç öğe içerir?

5 `{1, 2, 3} ^ {2, 3, 4}` nedir?

6 `remove` ile `discard` arasındaki fark nedir?

Cevaplar

1 `0` ve `KeyError`.

2 Değiştirilemez (immutable) tipler: string, sayı, tuple vb.

3 `{0: 0, 1: 1, 2: 2, 3: 0, 4: 1}`

4 4 öğe: `m`, `i`, `s`, `p`.

5 `{1, 4}`

6 Öğe kümede yoksa `remove` KeyError fırlatır, `discard` sessizce hiçbir şey yapmaz.

Bölüm 8 · Sınıflar ve Nesne Yönelimli Programlama

Bölümün amacı

Bu bölüm veriyi ve o veri üzerinde çalışan kodu tek bir **nesnede** birleştirmeyi öğretir. Sonunda şunları yapabilmelisin:

- Prosedürel programlama ile nesne yönelimli programlamayı (OOP) karşılaştırmak; kapsülleme ve veri gizlemeyi açıklamak
- `class`, `__init__`, `self` ile sınıf tanımlamak ve örnek (instance) oluşturmak
- Private öznitelikler, accessor/mutator metotlar ve `__str__` yazmak
- UML sınıf diyagramını okumak; bir problem açıklamasından sınıfları ve sorumluluklarını çıkarmak

Konu anlatımı

Prosedürel programlama (Slayt 1–3)

Prosedürel programlama, belirli görevleri yapan fonksiyonlardan (prosedürlerden) oluşur. Veri, prosedürlerden ayrı durur ve prosedürden prosedüre aktarılır; odak, veri üzerinde çalışan prosedürlerdir. Veri yapısı değişince onu kullanan tüm fonksiyonların değişmesi gerekebilir.

Nesne yönelimli programlama (Slayt 4–8)

OOP nesneler oluşturmaya odaklanır. **Nesne**, hem veri hem prosedür içerir:

- Nesnenin verisine **veri öznitelikleri**, prosedürlerine **metotlar** denir.
- **Kapsülleme:** veriyi ve kodu tek bir nesnede birleştirmek.
- **Veri gizleme:** nesnenin verisine yalnızca kendi metotları erişir; veri kazara bozulmaz ve dışarıdaki kodun iç yapıyı bilmesi gerekmez.
- **Yeniden kullanım:** aynı nesne farklı programlarda kullanılabilir.

Bir saat nesnesinde saniye/dakika/saat **veri öznitelikleridir**; `set_time` gibi **public metotlar** dışarıya açıktır, iç işleyiş için **private metotlar** kullanılır.

Sınıf ve örnek (Slayt 9–11)

Sınıf, bir nesne tipinin özniteliklerini ve metotlarını tanımlayan koddur (ev planı, kurabiye kalıbı). **Örnek** (instance) o sınıftan oluşturulmuş belirli bir nesnedir. Bir sınıfın birçok örneği olabilir.

Sınıf tanımı, self ve init (Slayt 12–15)

- `class SinifAdi:` ile başlar; ad genellikle büyük harfle yazılır.
- Her metodun ilk parametresi `self`'tir ve metodun üzerinde çalıştığı nesneyi gösterir.
- `__init__` başlatıcıdır: örnek oluşturulurken otomatik çalışır ve öznitelikleri başlatır.
- Örnek: `nesne = SinifAdi()` · metot çağrısı: `nesne.metot()`; `self` otomatik aktarılır.

PYTHON

```python
import random
random.seed(3)

class Coin:
    def __init__(self):
        self.sideup = 'Heads'

    def toss(self):
        if random.randint(0, 1) == 0:
            self.sideup = 'Heads'
        else:
            self.sideup = 'Tails'

    def get_sideup(self):
        return self.sideup

my_coin = Coin()
print('Başta:', my_coin.get_sideup())
for _ in range(4):
    my_coin.toss()
    print(my_coin.get_sideup(), end=' ')
```

ÇIKTI

```
Başta: Heads
Heads Heads Tails Tails
```

Gizli öznitelikler, parametreli metotlar ve str (Slayt 16–18)

Adı **iki alt çizgiyle** başlayan öznitelik (`self.__balance`) private olur ve dışarıdan doğrudan erişilemez. `__init__` ve diğer metotlar `self`'e ek parametreler alabilir. Bir nesnenin **durumu**, özniteliklerinin o anki değerleridir; `__str__` durumu anlatan bir string döndürür ve nesne `print` ya da `str` ile kullanılınca otomatik çağrılır. Sınıflar modüllerde (`.py`) saklanıp import edilebilir.

PYTHON

```python
class BankAccount:
    def __init__(self, bal):
        self.__balance = bal

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if self.__balance >= amount:
            self.__balance -= amount
        else:
            print('Error: Insufficient funds')

    def get_balance(self):
        return self.__balance

    def __str__(self):
        return f'The balance is ${self.__balance:,.2f}'

hesap = BankAccount(1000.0)
hesap.deposit(500)
hesap.withdraw(2000)
print(hesap.get_balance())
print(hesap)
```

ÇIKTI

```
Error: Insufficient funds
1500.0
The balance is $1,500.00
```

Örnek öznitelikleri (Slayt 19–21)

Bir metot `self.ad = ...` ile öznitelik oluşturduğunda bu **örnek özniteliğidir**: her örneğin **kendi** kopyası vardır. Bir nesneyi değiştirmek diğerlerini etkilemez.

PYTHON

```python
class Coin:
    def __init__(self):
        self.sideup = 'Heads'

coin1 = Coin()
coin2 = Coin()
coin1.sideup = 'Tails'
print(coin1.sideup, coin2.sideup)
```

ÇIKTI

```
Tails Heads
```

Accessor, mutator ve nesneyi argüman vermek (Slayt 22–23)

Öznitelikler genellikle private tutulur ve metotlarla erişilir:

- **Accessor** (erişici): değeri değiştirmeden döndürür (`get_balance`).
- **Mutator** (değiştirici): değeri saklar veya değiştirir (`set_balance`, `deposit`).

Bir nesneyi fonksiyona vermek ona bir **referans** vermektir: fonksiyon asıl nesnenin metotlarını çağırıp durumunu değiştirebilir.

PYTHON

```
class Coin:
    def __init__(self):
        self.__sideup = 'Heads'
    def set_tails(self):
        self.__sideup = 'Tails'
    def get_sideup(self):
        return self.__sideup

def cevir(coin_obj):
    coin_obj.set_tails()

c = Coin()
cevir(c)
print(c.get_sideup())
```

ÇIKTI

```
Tails
```

UML sınıf diyagramı (Slayt 24–26)

UML'de bir sınıf üç bölmeli bir kutudur: üstte **sınıf adı**, ortada **veri öznitelikleri**, altta **metotlar**.

Coin
`__sideup`
`__init__()`, `toss()`, `get_sideup()`

Problemdeki sınıfları bulmak (Slayt 27–32)

1. Problem alanının yazılı açıklamasını al.
2. Açıklamadaki tüm **isimleri** (isim tamlamaları, zamirler) belirle; her biri potansiyel sınıftır.
3. Listeyi **daralt**: aynı anlama gelenleri, programın ilgilenmediği öğeleri, sınıf değil nesne olanları ve basit bir değişkene sığan değerleri çıkar.

Sınıfın **sorumlulukları**: bilmesi gerekenler → veri öznitelikleri; yapması gerekenler → metotlar.

Sık yapılan hatalar

- **Metotta `self` parametresini unutmak:** çağrıda Python nesneyi otomatik gönderdiği için `TypeError` olur.

PYTHON
```
class Sayac:
    def __init__(self):
        self.n = 0
    def artir():
        self.n += 1

Sayac().artir()
```

ÇIKTI
```
TypeError: artir() takes 0 positional arguments but 1 was given
```

- **Private özniteliğe dışarıdan erişmeye çalışmak:** `hesap.__balance` → `AttributeError`; accessor kullan.

PYTHON
```
class Hesap:
    def __init__(self, b):
        self.__bakiye = b
print(Hesap(100).__bakiye)
```

ÇIKTI
```
AttributeError: 'Hesap' object has no attribute '__bakiye'
```

- **Özniteliği `self.` olmadan atamak:** `__init__` içinde `n = 0` yerel değişken olur, öznitelik oluşmaz.
- **Sınıfla örneği karıştırmak:** `Coin.toss()` değil, `my_coin = Coin()` sonra `my_coin.toss()`.

Kısa tekrar soruları

1 Kapsülleme ve veri gizleme ne demektir?

2 `__init__` ne zaman çalışır?

3 `self` parametresi neyi gösterir?

4 Accessor ve mutator metot örneği ver.

5 `print(nesne)` yazınca hangi metot otomatik çağrılır?

6 Bir problem açıklamasından sınıf bulurken hangi isimler listeden çıkarılır?

Cevaplar

1 Kapsülleme: veri ve kodu tek nesnede birleştirmek. Veri gizleme: verinin dışarıdan gizlenip yalnızca nesnenin metotlarıyla erişilebilmesi.

2 Sınıftan bir örnek oluşturulduğu anda otomatik olarak.

3 Metodun üzerinde çalıştığı belirli nesneyi (örneği).

4 Accessor: `get_balance()`; mutator: `deposit(amount)` veya `set_balance(x)`.

5 `__str__`

6 Aynı anlama gelenler, programın ilgilenmediği öğeler, sınıf değil nesne olanlar ve basit bir değişkene sığan değerler.

Bölüm 9 · Kalıtım ve Çok Biçimlilik

Bölümün amacı

Bu bölüm, genel bir sınıftan özelleşmiş sınıflar türetmeyi ve aynı metot çağrısının nesnenin tipine göre farklı davranmasını öğretir. Sonunda şunları yapabilmelisin:

- "Is a" ilişkisini, üst sınıf (superclass) ve alt sınıf (subclass) kavramlarını açıklamak
- Python'da alt sınıf tanımlamak ve alt sınıfın başlatıcısından üst sınıfın başlatıcısını çağırmak
- UML'de kalıtımı okumak
- Metotları geçersiz kılarak (override) çok biçimlilik kurmak ve `isinstance` kullanmak

Konu anlatımı

Genel ve özel nesneler (Slayt 1–4)

Gerçek dünyada birçok nesne daha genel bir nesnenin **özelleşmiş** halidir. Çekirge ve arı birer böcektir: böceklerin genel özelliklerine sahiptirler, ayrıca çekirge zıplar, arı sokar ve bal yapar.

"Is a" ilişkisi, üst sınıf ve alt sınıf (Slayt 5–7)

Bir nesne diğerinin özel haliyse aralarında **"is a"** (bir ... türüdür) ilişkisi vardır: dikdörtgen bir şekildir, papatya bir çiçektir. **Kalıtım** bu ilişkiyi sınıflar arasında kurar:

- **Üst sınıf** (superclass / base class): genel sınıf
- **Alt sınıf** (subclass / derived class): üst sınıfın öznitelik ve metotlarını **miras alır**, kendi yenilerini ekler

Örnek: araba, kamyonet ve SUV birer otomobildir. Marka, model yılı, kilometre ve fiyat **Automobile**'da; kapı sayısı (Car), çekiş tipi (Truck), yolcu kapasitesi (SUV) alt sınıflardadır.

Python'da alt sınıf ve UML (Slayt 8–10)

- Üst sınıf, alt sınıfın adından sonra parantez içinde yazılır: `class Car(Automobile):`
- Alt sınıfın `__init__`'i önce **üst sınıfın `__init__`'ini çağırır**, sonra kendi özniteliklerini başlatır.

- UML'de kalıtım, **alt sınıftan üst sınıfa** doğru içi boş üçgen uçlu bir okla gösterilir.

PYTHON
```python
class Automobile:
    def __init__(self, make, model, mileage, price):
        self.__make = make
        self.__model = model
        self.__mileage = mileage
        self.__price = price

    def get_make(self):
        return self.__make

    def get_price(self):
        return self.__price

class Car(Automobile):
    def __init__(self, make, model, mileage, price, doors):
        Automobile.__init__(self, make, model, mileage, price)
        self.__doors = doors

    def get_doors(self):
        return self.__doors

car = Car('BMW', 2001, 70000, 15000.0, 4)
print(car.get_make(), car.get_price(), car.get_doors())
```

ÇIKTI
```
BMW 15000.0 4
```

`get_make` ve `get_price` Car'da yazılmadı; Automobile'dan **miras** geldi.

Ek bilgi (resmi dokümantasyon): `Automobile.__init__(self, ...)` yerine `super().__init__(...)` da yazılabilir; `super()` üst sınıfı otomatik bulur ve `self`'i kendisi aktarır. Kaynak: docs.python.org/3/library/functions.html#super

Çok biçimlilik (Slayt 11–12)

Çok biçimlilik (polymorphism), bir nesnenin farklı biçimler alabilmesidir. İki parçası vardır:

1. Üst sınıftaki bir metodu alt sınıfta **aynı adla yeniden tanımlamak** (override).
2. Metot çağrıldığında, **nesnenin tipine göre** doğru sürümün çalışması.

Alt sınıftaki metot, üst sınıftakini çağırıp ekleme yapabilir ya da tamamen farklı bir şey yapabilir.

PYTHON

```python
class Mammal:
    def __init__(self, species):
        self.__species = species

    def show_species(self):
        print('I am a', self.__species)

    def make_sound(self):
        print('Grrrrr')

class Dog(Mammal):
    def __init__(self):
        Mammal.__init__(self, 'Dog')

    def make_sound(self):
        print('Woof! Woof!')

class Cat(Mammal):
    def __init__(self):
        Mammal.__init__(self, 'Cat')

    def make_sound(self):
        print('Meow')

for animal in [Mammal('regular mammal'), Dog(), Cat()]:
    animal.show_species()
    animal.make_sound()
```

ÇIKTI

```
I am a regular mammal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
```

Döngü her nesne için **aynı** `make_sound()` çağrısını yapar ama her nesne kendi sürümünü çalıştırır.

isinstance (Slayt 13–14)

Bir metot beklediği sınıftan olmayan bir nesne alırsa `AttributeError` oluşabilir. `isinstance(nesne, Sinif)` nesnenin o sınıfın (ya da alt sınıfının) örneği olup olmadığını söyler. **Alt sınıfın örneği, üst sınıfın da örneği sayılır.**

PYTHON

```python
class Mammal:
    def make_sound(self):
        print('Grrrrr')

class Dog(Mammal):
    pass

def sesini_goster(nesne):
    if isinstance(nesne, Mammal):
        nesne.make_sound()
    else:
        print('Bu bir memeli değil!')

d = Dog()
print(isinstance(d, Dog), isinstance(d, Mammal), isinstance(Mammal(), Dog))
sesini_goster(d)
sesini_goster('köpek')
```

ÇIKTI

```
True True False
Grrrrr
Bu bir memeli değil!
```

Sık yapılan hatalar

- **Alt sınıfın `__init__`'inde üst sınıfın `__init__`'ini çağırmamak:** üst sınıfın öznitelikleri hiç oluşmaz.

PYTHON

```python
class A:
    def __init__(self):
        self.x = 1

class B(A):
    def __init__(self):
        self.y = 2

print(B().x)
```

ÇIKTI

```
AttributeError: 'B' object has no attribute 'x'
```

- **Override için farklı bir ad kullanmak:** alt sınıftaki metot **aynı adı** taşımalıdır; yoksa yeni bir metot olur.
- **Kalıtım okunu ters çizmek:** UML'de ok alt sınıftan üst sınıfa doğrudur.

- `isinstance`**'ın yönünü karıştırmak:** üst sınıfın örneği alt sınıfın örneği **değildir**.

Kısa tekrar soruları

1 "Is a" ilişkisine iki örnek ver.

2 Alt sınıf üst sınıftan neyi miras alır?

3 `class Truck(Automobile):` satırında hangisi üst sınıftır?

4 Override nedir? Çok biçimlilikle ilişkisi nedir?

5 `Dog`, `Mammal`'ın alt sınıfıysa `isinstance(Dog(), Mammal)` ve `isinstance(Mammal(), Dog)` ne döndürür?

Cevaplar

1 Örneğin: araba bir otomobildir; papatya bir çiçektir (ya da arı bir böcektir).

2 Üst sınıfın öznitelik ve metotlarını.

3 `Automobile`.

4 Alt sınıfta üst sınıftaki metotla aynı adlı bir metot tanımlamaktır. Çok biçimlilikte aynı çağrı, nesnenin tipine göre bu farklı sürümlerden doğru olanı çalıştırır.

5 `True` ve `False`.

Bölüm 10 · Özyineleme

Bölümün amacı

Bu bölüm kendini çağıran fonksiyonlarla problem çözmeyi öğretir. Sonunda şunları yapabilmelisin:

- Özyinelemeli fonksiyonun ve özyineleme derinliğinin ne olduğunu açıklamak
- Bir problemi **temel durum** ve **özyinelemeli durum** olarak ayırmak
- Faktöriyel, liste aralığı toplamı, Fibonacci, EBOB ve Hanoi kuleleri algoritmalarını yazmak
- Doğrudan ve dolaylı özyinelemeyi ayırt etmek; özyineleme ile döngüyü karşılaştırmak

Konu anlatımı

Kendini çağıran fonksiyon (Slayt 1–5)

Özyinelemeli (recursive) fonksiyon kendini çağırır. Kaç kez tekrar edeceğini kontrol eden bir yolu olmalıdır; bu genellikle "ne zaman değer döndüreceğim, ne zaman kendimi çağıracağım" kararını veren bir `if-else`'tir. Fonksiyonun kendini kaç kez çağırdığına **özyineleme derinliği** denir. Bir çağrı bitince kontrol, o çağrının hemen sonrasına döner.

PYTHON
```python
def message(times):
    if times > 0:
        print('This is a recursive function.', times)
        message(times - 1)
    print('dönüş:', times)

message(3)
```

ÇIKTI
```
This is a recursive function. 3
This is a recursive function. 2
This is a recursive function. 1
dönüş: 0
dönüş: 1
dönüş: 2
dönüş: 3
```

Çıktıda "dönüş" satırlarının **ters sırada** gelmesi, her çağrının bitince kendisini çağıranın kaldığı yere döndüğünü gösterir.

Özyinelemeyle problem çözmek (Slayt 6–7)

Özyineleme güçlü ama **zorunlu değildir**: özyinelemeyle çözülen her problem döngüyle de çözülebilir. Her çağrının ek yükü olduğu için özyinelemeli algoritmalar genellikle **daha az verimlidir**. Genel kalıp:

- **Temel durum (base case):** problem özyineleme olmadan hemen çözülebiliyorsa çöz ve döndür.
- **Özyinelemeli durum:** değilse problemi aynı yapıda **daha küçük** bir probleme indirge ve fonksiyonu tekrar çağır.

Faktöriyel (Slayt 8–11)

n = 0 için n! = 1 (temel durum); n > 0 için n! = n × (n − 1)! (özyinelemeli durum). Her çağrıda parametre bir azalır ve sonunda temel duruma ulaşılır.

PYTHON

```python
def factorial(num):
    if num == 0:
        return 1
    else:
        return num * factorial(num - 1)

print(factorial(4), factorial(0), factorial(10))
```

ÇIKTI

```
24 1 3628800
```

`factorial(4)` = 4 × factorial(3) = 4 × 3 × factorial(2) = … = 4 × 3 × 2 × 1 × 1 = 24.

Doğrudan ve dolaylı özyineleme (Slayt 12)

Doğrudan özyinelemede fonksiyon kendini çağırır. **Dolaylı** özyinelemede A, B'yi; B de A'yı çağırır.

PYTHON

```python
def cift_mi(n):
    if n == 0:
        return True
    return tek_mi(n - 1)

def tek_mi(n):
    if n == 0:
        return False
    return cift_mi(n - 1)

print(cift_mi(4), tek_mi(7), cift_mi(3))
```

ÇIKTI

```
True True False
```

Liste aralığını toplamak (Slayt 13–14)

Temel durum: başlangıç indeksi bitişi geçtiyse 0. Özyinelemeli durum: o anki eleman + kalan aralığın toplamı.

PYTHON

```python
def range_sum(num_list, start, end):
    if start > end:
        return 0
    else:
        return num_list[start] + range_sum(num_list, start + 1, end)

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(range_sum(numbers, 2, 5))
```

ÇIKTI

```
18
```

Fibonacci ve EBOB (Slayt 15–16)

Fibonacci serisinin **iki** temel durumu vardır: Fib(0) = 0, Fib(1) = 1; n > 1 için Fib(n) = Fib(n−1) + Fib(n−2).

EBOB (GCD): x, y'ye tam bölünüyorsa gcd(x, y) = y; değilse gcd(x, y) = gcd(y, x % y).

PYTHON

```python
def fib(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)

def gcd(x, y):
    if x % y == 0:
        return y
    else:
        return gcd(y, x % y)

print([fib(i) for i in range(10)])
print(gcd(49, 28), gcd(12, 18))
```

ÇIKTI

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
7 6
```

Hanoi kuleleri (Slayt 17–21)

Üç çubuk ve küçülen diskler. Kurallar: aynı anda **yalnızca bir disk** taşınır, büyük disk küçüğün üstüne konamaz, taşınan disk dışında tüm diskler bir çubukta durur. n diski 1'den 3'e taşımak için:

1. n − 1 diski 1'den 2'ye taşı (3'ü geçici kullanarak),
2. kalan en büyük diski 1'den 3'e taşı,
3. n − 1 diski 2'den 3'e taşı (1'i geçici kullanarak).

PYTHON

```python
def move_discs(num, from_peg, to_peg, temp_peg):
    if num > 0:
        move_discs(num - 1, from_peg, temp_peg, to_peg)
        print('Move a disc from peg', from_peg, 'to peg', to_peg)
        move_discs(num - 1, temp_peg, to_peg, from_peg)

move_discs(3, 1, 3, 2)
```

ÇIKTI

```
Move a disc from peg 1 to peg 3
Move a disc from peg 1 to peg 2
Move a disc from peg 3 to peg 2
Move a disc from peg 1 to peg 3
Move a disc from peg 2 to peg 1
Move a disc from peg 2 to peg 3
Move a disc from peg 1 to peg 3
```

Üç disk **7** hamlede taşınır (genel olarak $2^n - 1$ hamle).

Özyineleme mi, döngü mü? (Slayt 22–23)

Özyineleme kullanmamanın nedenleri: fonksiyon çağrısı yükü yüzünden daha az verimlidir ve döngülü çözüm çoğu zaman daha anlaşılırdır. Ama tanımı zaten özyinelemeli olan bazı problemler (Fibonacci, Hanoi) özyinelemeyle daha doğal yazılır.

PYTHON

```python
def factorial_dongu(n):
    sonuc = 1
    for i in range(1, n + 1):
        sonuc *= i
    return sonuc
```

PYTHON (DEVAM)

```
print(factorial_dongu(4), factorial_dongu(10))
```

ÇIKTI

```
24 3628800
```

Sık yapılan hatalar

- **Temel durumu unutmak ya da ona hiç ulaşamamak:** fonksiyon kendini sonsuza kadar çağırır, Python `RecursionError` ile durdurur.

PYTHON

```
def geri_say(n):
    return geri_say(n - 1)

geri_say(5)
```

ÇIKTI

```
RecursionError: maximum recursion depth exceeded
```

- **Problemi küçültmemek:** özyinelemeli çağrı parametreyi temel duruma doğru değiştirmelidir (`n - 1` gibi).
- **Fibonacci'nin ikinci temel durumunu unutmak:** `fib(1)` için ayrı bir dönüş olmazsa `fib(-1)`'e iner.
- `return`**'ü unutmak:** `num * factorial(num - 1)` döndürülmezse sonuç `None` olur.

Kısa tekrar soruları

1 Temel durum ve özyinelemeli durum nedir?

2 `factorial(3)` çağrısında özyineleme derinliği kaçtır (temel durum dahil kaç çağrı yapılır)?

3 `range_sum([5, 1, 7, 3], 1, 3)` nedir?

4 `gcd(18, 12)` adım adım nasıl hesaplanır?

5 4 diskli Hanoi kulesi kaç hamlede çözülür?

6 Özyinelemeli çözümlerin döngülü çözümlere göre bir dezavantajı nedir?

Cevaplar

1 Temel durum, özyineleme olmadan doğrudan çözülen durumdur; özyinelemeli durum, problemi daha küçük bir kopyasına indirgeyip fonksiyonu yeniden çağırır.

2 4 çağrı: `factorial(3)`, `(2)`, `(1)`, `(0)`.

3 1 + 7 + 3 = 11.

4 18 % 12 = 6 ≠ 0 → gcd(12, 6); 12 % 6 = 0 → 6.

5 $2^4 - 1 = 15$ hamle.

6 Her fonksiyon çağrısının ek yükü olduğu için genellikle daha az verimlidir (ve çoğu zaman daha zor anlaşılır).

Bölüm 11 · Veritabanı Programlama

Bölümün amacı

Veri büyüdükçe düz dosyalar yetersiz kalır. Bu bölüm Python'dan **SQLite** veritabanıyla çalışmayı öğretir. Sonunda şunları yapabilmelisin:

- DBMS, SQL, tablo, satır, sütun, birincil anahtar ve kimlik sütunu kavramlarını açıklamak
- `sqlite3` ile bağlanmak, tablo oluşturmak/silmek ve `INSERT` ile veri eklemek
- `SELECT`, `WHERE`, `LIKE`, `ORDER BY` ve toplama fonksiyonlarıyla sorgulamak
- `UPDATE` ve `DELETE` kullanmak; veritabanı hatalarını `try/except` ile yakalamak
- İlişkisel tasarım, yabancı anahtar ve birden çok tablodan sorgu yazmak (CRUD)

Konu anlatımı

Veritabanı yönetim sistemi (Slayt 1–5)

Klasik dosyalar az veri için yeterlidir ama veri büyüdükçe basit işlemler bile zahmetli ve verimsiz olur. **DBMS** (veritabanı yönetim sistemi) büyük veriyle düzenli ve verimli çalışmak için tasarlanmış yazılımdır. Python uygulaması veriye doğrudan değil DBMS aracılığıyla erişir: komut gönderir → DBMS çalıştırır → sonucu geri yollar.

SQL ve SQLite (Slayt 6–7)

SQL DBMS'lerle çalışmanın standart dilidir; genel amaçlı bir programlama dili değildir. SQL ifadeleri Python'dan **string olarak** gönderilir. Derste kullanılan **SQLite**, Python ile birlikte `sqlite3` modülünde gelir.

Tablolar, satırlar, sütunlar ve veri tipleri (Slayt 8–11)

Veritabanı tablolardan oluşur. **Satır** bir öğenin tüm verisi, **sütun** tek bir veri parçasıdır.

SQLite tipi	Açıklama	Python karşılığı
NULL	bilinmeyen değer	`None`
INTEGER	tam sayı	`int`
REAL	gerçek sayı	`float`
TEXT	metin	`str`
BLOB	ikili büyük nesne	herhangi bir nesne

Birincil anahtar, kimlik sütunu, NULL (Slayt 12–14)

Birincil anahtar her satırda benzersiz değer tutan ve satırı tanımlayan sütundur; **asla NULL olamaz**. **Kimlik sütunu**, DBMS'in ürettiği (genellikle otomatik artan) benzersiz değerlerdir; tabloda benzersiz bir sütun yoksa birincil anahtar yapılır. `NOT NULL` kısıtı bir sütunun boş kalmasını engeller.

Bağlanma ve tablo oluşturma (Slayt 15–22)

Tipik süreç: **bağlan** (`sqlite3.connect`) → **cursor al** → işlemleri `cur.execute(sql)` ile yap → **commit** → **kapat**. `connect` veritabanı yoksa oluşturur (`':memory:'` geçici, bellekte bir veritabanıdır).

`CREATE TABLE Ad (Sutun Tip, ...)` tablo oluşturur; SQLite'ta `INTEGER PRIMARY KEY` sütunu otomatik artan kimlik sütunu olur. `IF NOT EXISTS` / `DROP TABLE IF EXISTS` güvenli oluşturma/silme sağlar.

PYTHON

```python
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('''CREATE TABLE IF NOT EXISTS Inventory (
                    ItemID INTEGER PRIMARY KEY NOT NULL,
                    ItemName TEXT,
                    Price REAL)''')
conn.commit()
cur.execute("SELECT name FROM sqlite_master WHERE type = 'table'")
print(cur.fetchall())
conn.close()
```

ÇIKTI

```
[('Inventory',)]
```

INSERT ile veri eklemek (Slayt 23–27)

`INSERT INTO Tablo (Sutunlar) VALUES (Degerler)`. Kimlik sütunu yazılmazsa SQLite değeri kendisi üretir; tek ifadede birden çok satır eklenebilir. Değişkenlerin değerleri `?` **yer tutucularıyla** verilir (string birleştirmeyle değil).

PYTHON

```python
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('CREATE TABLE Inventory (ItemID INTEGER PRIMARY KEY NOT NULL,
↪ ItemName TEXT, Price REAL)')
cur.execute("INSERT INTO Inventory (ItemName, Price) VALUES ('Screwdriver',
↪ 4.99), ('Hammer', 12.99)")
item_name, price = 'Wrench', 16.99
```

PYTHON (DEVAM)

```
cur.execute('INSERT INTO Inventory (ItemName, Price) VALUES (?, ?)', (
↪ item_name, price))
conn.commit()
cur.execute('SELECT * FROM Inventory')
print(cur.fetchall())
conn.close()
```

ÇIKTI

```
[(1, 'Screwdriver', 4.99), (2, 'Hammer', 12.99), (3, 'Wrench', 16.99)]
```

Ek bilgi (resmi dokümantasyon): `?` yer tutucuları, değerleri SQL metnine elle eklemenin yol açabileceği SQL enjeksiyonu saldırılarını önler. Kaynak: docs.python.org/3/library/sqlite3.html (How to use placeholders to bind values in SQL queries).

SELECT ve fetch (Slayt 28–32)

`SELECT Sutunlar FROM Tablo` (`*` tüm sütunlar). Python'da iki adımlıdır: önce `execute`, sonra sonuçları getir: `fetchall()` tüm satırları liste olarak, `fetchone()` her çağrıda bir satırı tuple olarak (kalmayınca `None`) döndürür.

PYTHON

```
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('CREATE TABLE Products (Description TEXT, RetailPrice REAL,
↪ UnitsOnHand INTEGER)')
cur.execute("""INSERT INTO Products VALUES ('Dark Chocolate Bar', 2.99, 120),
               ('Chocolate Truffles', 12.99, 40), ('Mint Bar', 3.49, 80)""")
cur.execute('SELECT Description, RetailPrice FROM Products')
for row in cur.fetchall():
    print(f'{row[0]:20} {row[1]:6.2f}')
cur.execute('SELECT Description FROM Products')
print(cur.fetchone(), cur.fetchone())
```

ÇIKTI

```
Dark Chocolate Bar     2.99
Chocolate Truffles    12.99
Mint Bar               3.49
('Dark Chocolate Bar',) ('Chocolate Truffles',)
```

WHERE, LIKE, ORDER BY ve toplama fonksiyonları (Slayt 33–42)

- `WHERE kriter` yalnızca kritere uyan satırları getirir; `>`, `<`, `=`/`==`, `!=`/`<>`, `AND`, `OR`, `NOT` kullanılabilir.
- `LIKE` alt string arar; `%` birden çok karakter yerine geçer: `LIKE '%Bar%'`.

- `ORDER BY sutun` sıralar (varsayılan **artan**; azalan için `DESC`).
- `AVG`, `SUM`, `MIN`, `MAX`, `COUNT(*)` toplama fonksiyonlarıdır.

PYTHON
```python
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('CREATE TABLE Products (Description TEXT, RetailPrice REAL,
↪ UnitsOnHand INTEGER)')
cur.execute("""INSERT INTO Products VALUES ('Dark Bar', 2.99, 120), (
↪ 'Truffles', 12.99, 40),
               ('Mint Bar', 3.49, 80), ('Caramel', 9.50, 20)""")
cur.execute("SELECT Description FROM Products WHERE Description LIKE '%Bar%'
↪ ORDER BY RetailPrice")
print(cur.fetchall())
cur.execute('SELECT Description FROM Products WHERE RetailPrice > 3.00 AND
↪ UnitsOnHand < 100 ORDER BY RetailPrice DESC')
print(cur.fetchall())
cur.execute('SELECT COUNT(*), MIN(RetailPrice), MAX(RetailPrice), ROUND(AVG(
↪ RetailPrice), 2) FROM Products')
print(cur.fetchone())
```

ÇIKTI
```
[('Dark Bar',), ('Mint Bar',)]
[('Truffles',), ('Caramel',), ('Mint Bar',)]
(4, 2.99, 12.99, 7.24)
```

UPDATE ve DELETE (Slayt 43–52)

`UPDATE Tablo SET Sutun = Deger WHERE kriter` var olan satırları değiştirir; `DELETE FROM Tablo WHERE kriter` siler. Etkilenen satır sayısı `cur.rowcount`'tadır. `WHERE` **unutulursa tablodaki tüm satırlar** güncellenir/silinir.

PYTHON
```python
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('CREATE TABLE Products (Description TEXT, Units INTEGER)')
cur.execute("INSERT INTO Products VALUES ('Mint Bar', 5), ('Truffles', 8), (
↪ 'Dark Bar', 2)")
cur.execute("UPDATE Products SET Units = 0 WHERE Description LIKE '%Bar%'")
print('güncellenen:', cur.rowcount)
cur.execute('DELETE FROM Products WHERE Units == 0')
print('silinen:', cur.rowcount)
conn.commit()
cur.execute('SELECT * FROM Products')
print(cur.fetchall())
```

ÇIKTI

```
güncellenen: 2
silinen: 2
[('Truffles', 8)]
```

Birincil anahtarlar hakkında daha fazlası (Slayt 53–57)

Bir tablonun **tek** birincil anahtarı olur ama birden çok sütun birleşip **bileşik anahtar** oluşturabilir: `PRIMARY KEY(RoomNumber, Building)`. SQLite tablolarında otomatik artan bir **RowID** vardır; `INTEGER PRIMARY KEY` aslında onun takma adıdır. INTEGER olmayan birincil anahtarda değeri sen vermeli ve `NOT NULL` kullanmalısın.

Veritabanı hatalarını işlemek ve CRUD (Slayt 58–60)

`sqlite3.Error` tüm veritabanı hatalarının temel sınıfıdır. Veritabanı kodu `try/except/finally` ile yazılır; bağlantı `finally`'de kapatılır. Bir veritabanı uygulamasının dört temel işlemi **CRUD**'dur: Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE).

PYTHON

```python
import sqlite3
conn = None
try:
    conn = sqlite3.connect(':memory:')
    cur = conn.cursor()
    cur.execute('CREATE TABLE T (ID INTEGER PRIMARY KEY, Ad TEXT)')
    cur.execute("INSERT INTO T VALUES (1, 'a')")
    cur.execute("INSERT INTO T VALUES (1, 'b')")
except sqlite3.Error as err:
    print('Veritabanı hatası:', type(err).__name__, '-', err)
finally:
    if conn != None:
        conn.close()
```

ÇIKTI

```
Veritabanı hatası: IntegrityError - UNIQUE constraint failed: T.ID
```

İlişkisel veri ve yabancı anahtarlar (Slayt 61–73)

Aynı verinin (ör. "Marketing, New York City") birçok satırda tekrarlanması depolamayı boşa harcar ve **tutarsızlığa** yol açar. **İlişkisel tasarımda** veri tablolara bölünür: Employees tablosu departman adı yerine `DepartmentID` tutar.

Yabancı anahtar, başka bir tablonun birincil anahtarını referans eden sütundur ve **referans bütünlüğü** sağlar. ERD'de 1 → ∞ **bire-çok** ilişkiyi gösterir (bir departmanda çok çalışan). SQLite yabancı anahtarları **varsayılan olarak**

zorlamaz; `PRAGMA foreign_keys=ON` gerekir.

PYTHON
```python
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('PRAGMA foreign_keys=ON')
cur.execute('CREATE TABLE Departments(DepartmentID INTEGER PRIMARY KEY NOT
↪ NULL, DepartmentName TEXT)')
cur.execute('''CREATE TABLE Employees(EmployeeID INTEGER PRIMARY KEY NOT NULL,
↪ Name TEXT,
               DepartmentID INTEGER,
               FOREIGN KEY(DepartmentID) REFERENCES Departments(
↪ DepartmentID))''')
cur.execute("INSERT INTO Departments VALUES (1, 'Marketing')")
cur.execute("INSERT INTO Employees VALUES (1, 'Jack Smith', 1)")
try:
    cur.execute("INSERT INTO Employees VALUES (2, 'Renee', 99)")
except sqlite3.Error as err:
    print('Reddedildi:', err)
```

ÇIKTI
```
Reddedildi: FOREIGN KEY constraint failed
```

Birden çok tablodan sorgu (Slayt 74–76)

Sütunlar `Tablo.Sutun` biçiminde **tam nitelikli adlarla** yazılır ve tablolar `WHERE` ile anahtarlar üzerinden eşleştirilir.

PYTHON
```python
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('CREATE TABLE Departments(DepartmentID INTEGER PRIMARY KEY,
↪ DepartmentName TEXT)')
cur.execute('CREATE TABLE Locations(LocationID INTEGER PRIMARY KEY, City
↪ TEXT)')
cur.execute('CREATE TABLE Employees(EmployeeID INTEGER PRIMARY KEY, Name TEXT,
↪ DepartmentID INTEGER, LocationID INTEGER)')
cur.execute("INSERT INTO Departments VALUES (1, 'Marketing'), (2,
↪ 'Accounting')")
cur.execute("INSERT INTO Locations VALUES (1, 'New York City'), (2, 'Boston')")
cur.execute("INSERT INTO Employees VALUES (1, 'Jack', 1, 1), (2, 'Sonia', 2,
↪ 2), (3, 'Renee', 1, 2)")
cur.execute('''SELECT Employees.Name, Departments.DepartmentName,
↪ Locations.City
               FROM Employees, Departments, Locations
               WHERE Employees.DepartmentID == Departments.DepartmentID AND
                     Employees.LocationID == Locations.LocationID''')
for satir in cur.fetchall():
    print(satir)
```

ÇIKTI

```
('Jack', 'Marketing', 'New York City')
('Sonia', 'Accounting', 'Boston')
('Renee', 'Marketing', 'Boston')
```

Sık yapılan hatalar

- `commit()` **yapmamak:** dosya tabanlı veritabanında değişiklikler kalıcı olmaz.
- `WHERE`**'siz** `UPDATE` **/** `DELETE`**:** tüm tablo etkilenir.
- **Var olmayan tabloyu sorgulamak →** `OperationalError`.

PYTHON

```python
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('CREATE TABLE Products (Name TEXT)')
cur.execute('SELECT * FROM Urunler')
```

ÇIKTI

```
sqlite3.OperationalError: no such table: Urunler
```

- **Birincil anahtar değerini tekrar etmek →** `IntegrityError`.
- **Değerleri string birleştirmeyle SQL'e eklemek:** `?` yer tutucusu kullan.
- **SQLite'ın yabancı anahtarı kendiliğinden denetlediğini sanmak:** `PRAGMA foreign_keys=ON` gerekir.

Kısa tekrar soruları

1. SQLite'ta `INTEGER PRIMARY KEY` sütunun özelliği nedir?
2. `fetchone()` satır kalmayınca ne döndürür?
3. `WHERE Description LIKE '%Bar'` hangi satırları seçer?
4. `ORDER BY Price` hangi yönde sıralar?
5. CRUD harfleri hangi SQL komutlarına karşılık gelir?
6. Yabancı anahtar ne işe yarar? Veri tekrarının iki sorunu nedir?

Cevaplar

1. Otomatik artan kimlik sütunu olur (RowID'nin takma adı); değer verilmezse SQLite üretir.
2. `None`
3. Açıklaması "Bar" ile **biten** satırları.
4. Artan (en küçük önce).

5 Create → INSERT, Read → SELECT, Update → UPDATE, Delete → DELETE.

6 Başka bir tablonun birincil anahtarını referans ederek tablolar arasında ilişki ve referans bütünlüğü kurar. Veri tekrarı depolamayı boşa harcar ve tutarsız/çelişkili veriye yol açar.

Bölüm 12 · Python Ortamları

Bölümün amacı

Veri bilimi çalışmaları için Python'u, paketleri ve araçları yönetmeyi kolaylaştıran üç ortamı tanıtır: **Anaconda Navigator**, **Jupyter Notebook** ve **Google Colab**. Sonunda şunları yapabilmelisin:

- Anaconda, conda ve Navigator'ın ne olduğunu ve birbirleriyle ilişkisini açıklamak
- "Ortam" (environment) kavramını ve neden gerektiğini anlatmak
- Jupyter'da code ve Markdown hücreleriyle çalışmak
- Google Colab'ın avantajlarını, sınırlamalarını ve ne zaman tercih edileceğini bilmek

Bu bölüm araç tanıtımıdır. Kodlar ağırlıklı olarak komut satırı ve notebook komutlarıdır; bu yüzden bu özette çalıştırılan Python örneği azdır.

Konu anlatımı

Anaconda nedir? (Slayt 1–3)

Anaconda, Python ve R için ücretsiz, açık kaynaklı bir **dağıtımdır**; veri bilimi, makine öğrenmesi ve bilimsel hesaplama için tasarlanmıştır. **conda** paket yöneticisi, Python'u NumPy, Pandas, Matplotlib gibi önceden kurulu birçok bilimsel kütüphaneyle birlikte getirir.

Anaconda Navigator bu dağıtımla gelen **grafik arayüzdür** (GUI); kütüphaneleri yönetirken komut satırına bağımlılığı azaltır.

Navigator'ın temel özellikleri (Slayt 4)

- **Uygulama başlatıcı:** Jupyter Notebook, JupyterLab, Spyder, VS Code, RStudio gibi araçları tek tıkla açar.
- **Ortam yönetimi:** ortamları görsel olarak oluşturur, kopyalar (clone) ve siler.
- **Paket yönetimi:** bir ortamda paket arar, kurar, günceller, kaldırır.

Ortam, belirli bir proje için belirli paket sürümlerinin kurulduğu **yalıtılmış alandır**. Örneğin Project_A Python 3.8, Project_B Python 3.10 kullanabilir. Ortamlar projeler arasındaki **paket çakışmalarını** önler.

```bash
conda create -n my-new-project python=3.10
conda activate my-new-project
conda install seaborn
```

Ek bilgi (resmi dokümantasyon): Yukarıdaki komutlar Navigator'daki "Create" ve "Apply" adımlarının komut satırı karşılığıdır. Kaynak: docs.conda.io (Managing environments).

Kurulum ve kısa demo (Slayt 5–7)

anaconda.com/download adresinden işletim sistemine uygun yükleyici indirilir; kurulum büyüktür çünkü çok paket içerir. Navigator **ayrıca kurulmaz**, dağıtıma dahildir.

- **Jupyter'ı açmak:** Home → Jupyter Notebook → Launch
- **Yeni ortam:** Environments → Create → ad ve Python sürümü → Create
- **Paket kurmak:** ortamı seç → filtreyi "Not Installed" yap → paketi ara (ör. seaborn) → işaretle → Apply

Özetle: **conda** çekirdekteki komut satırı paket yöneticisi, **Anaconda** hepsi bir arada dağıtım, **Navigator** bu araç kutusunun grafik arayüzüdür.

Jupyter Notebook nedir? (Slayt 8–9)

Canlı kod, görselleştirme ve anlatımı tek bir belgede birleştiren, açık kaynaklı bir web uygulamasıdır. Notebook'lar çalıştırılabilir kod (Julia, Python, R), Markdown metni, denklemler ve grafikler içerebilir.

- **Code hücresi:** kod yazılır, **Shift + Enter** ile çalıştırılır; çıktı hemen altında görünür.
- **Markdown hücresi:** `# Başlık`, `**kalın**`, `*italik*`, listeler; çalıştırılınca biçimli metne dönüşür.

Notebook'ta code hücresinin son satırındaki ifadenin değeri, `print` yazmadan da gösterilir:

```python
sayilar = [3, 1, 2]
print('Hello, World!')
sorted(sayilar)   # notebook'ta bu değer hücrenin altında görünür
print(sorted(sayilar))
```

```
Hello, World!
[1, 2, 3]
```

Özellikler ve kurulum (Slayt 10–11)

- **Etkileşimli iş akışı:** küçük parçaları tek tek çalıştırıp sonucu görmek; keşifsel veri analizi (EDA) için idealdir.
- **Hikâyecilik ve paylaşım:** veriyi yüklemeden sonuca kadar tüm süreç tek belgede.
- **Tekrarlanabilirlik:** kod, çıktı ve açıklama bir arada.
- **Dilden bağımsız:** hesaplamayı yapan **kernel**'lar sayesinde 40'tan fazla dil.

Kurulum: önerilen yol Anaconda (Jupyter dahildir). Minimal yol: `pip install notebook`, ardından `jupyter notebook`.

Kısa demo (Slayt 12–13)

Navigator'dan Launch → tarayıcıda **Jupyter Dashboard** açılır → **New → Python 3** → hücreye `print("Hello, World!")` yazıp Shift + Enter → hücre tipini **Markdown** yapıp `# My First Analysis` → Shift + Enter. Başlığa tıklayıp adlandır; dosya uzantısı `.ipynb`'dir, Ctrl + S ile kaydedilir.

Google Colab nedir? (Slayt 14–15)

Google Colaboratory, **kurulum gerektirmeyen**, bulut tabanlı ve ücretsiz bir Jupyter ortamıdır.

- **Sıfır kurulum:** yalnızca tarayıcı ve Google hesabı gerekir.
- **Bulut tabanlı:** notebook'lar Google Drive'a kaydedilir.
- **Ücretsiz donanım:** derin öğrenme için **GPU** ve **TPU** erişimi.
- **Kolay iş birliği:** Google Doc gibi paylaşılır, aynı anda düzenlenir.
- **Önceden kurulu kütüphaneler:** TensorFlow, PyTorch, pandas…; diğerleri `!pip install paket` ile.

Sınırlamalar (Slayt 16)

- **İnternet gerekir**; çevrimdışı çalışmaz.
- **Kaynak sınırları:** oturum kalıcı değildir; hareketsizlikten (ör. 90 dk) ya da azami süreden (ör. 12 saat) sonra kapanır. **Kurulan paketler ve Drive dışındaki dosyalar silinir.** En iyi GPU'lar garanti değildir.
- **Veri depolama:** veri Google Drive'a (mount ederek) kaydedilmeli ya da her oturumda yeniden yüklenmelidir.

Başlamak ve ne zaman kullanmalı? (Slayt 17–19)

colab.research.google.com → **New notebook**. GPU için: **Runtime → Change runtime type → Hardware accelerator: GPU (veya TPU) → Save**. Arayüz

Jupyter ile neredeyse aynıdır. Komut satırı komutları başa `!` konarak çalıştırılır:

```bash
!pip install scikit-learn
!nvidia-smi
```

Colab'ı şu durumlarda seç: ücretsiz güçlü donanım (GPU/TPU) gerektiğinde, kolay iş birliği istediğinde, sıfır kurulumla hızlı başlamak istediğinde; ama yalnızca oturum süre sınırları ve çevrimiçi olma şartı sorun değilse.

Sık yapılan hatalar

- **Navigator'ı ayrı bir program sanmak:** Anaconda dağıtımına dahildir.
- **Tüm projeleri tek ortamda çalıştırmak:** farklı sürüm isteyen paketler çakışır; her projeye ortam aç.
- **Markdown hücresine Python kodu yazıp çalışmasını beklemek:** Markdown hücreleri kodu çalıştırmaz, metin olarak biçimlendirir.
- **Colab'da kurulan paketlerin kalıcı olduğunu sanmak:** oturum kapanınca silinir; her oturumda yeniden kurulur.
- **Colab'da veriyi yalnızca çalışma ortamına yüklemek:** oturum bitince kaybolur; Drive'a kaydet.

Kısa tekrar soruları

1. conda, Anaconda ve Anaconda Navigator arasındaki ilişki nedir?
2. "Ortam" (environment) neden kullanılır?
3. Jupyter'da bir hücre nasıl çalıştırılır? Notebook dosyalarının uzantısı nedir?
4. Jupyter'daki kernel ne işe yarar?
5. Colab'ın iki avantajını ve iki sınırlamasını say.
6. Colab'da `pip install` komutunun başına ne konur?

Cevaplar

1. conda komut satırı paket yöneticisidir; Anaconda conda'yı ve birçok bilimsel paketi içeren dağıtımdır; Navigator bu dağıtımın grafik arayüzüdür.
2. Her projeye kendi paket sürümlerini yalıtılmış biçimde vererek paket çakışmalarını önlemek için.
3. Shift + Enter; uzantı `.ipynb`.
4. Kodun perde arkasında çalıştığı hesaplama motorudur; farklı kernel'lar farklı dilleri destekler.

5 Avantaj: sıfır kurulum, ücretsiz GPU/TPU, kolay iş birliği. Sınırlama: internet gerekir, oturum süreli ve kalıcı değildir (paketler ve geçici dosyalar silinir).

6 Ünlem işareti: `!pip install paket`.

Bölüm 13 · NumPy Bölüm 1

Bölümün amacı

Bu bölüm veri biliminin temel paketi **NumPy**'ı ve onun çekirdek veri yapısı **ndarray**'i öğretir. Sonunda şunları yapabilmelisin:

- Python listelerinin sayısal hesaplamadaki sorununu ve NumPy'ın neden hızlı olduğunu açıklamak
- Vektörleştirmeyi kullanmak; NumPy'ın ne zaman avantajlı olmadığını bilmek
- `ndim`, `shape`, `size`, `dtype`, `itemsize` özniteliklerini okumak
- `np.array`, `zeros`, `ones`, `empty`, `arange`, `linspace`, `eye` ile dizi oluşturmak
- 1, 2 ve 3 boyutlu dizileri indekslemek, dilimlemek ve `reshape` etmek
- **view** ile **copy** arasındaki farkı ve `.copy()`'nin ne zaman gerektiğini açıklamak

Konu anlatımı

Python listesinin sorunu (Slayt 1–3)

Python listesi esnek bir kaptır ve farklı tipte (heterojen) değerler tutabilir. Ama verinin kendisini değil, bellekte **dağınık duran nesnelere işaret eden pointer'ları** saklar. Liste üzerinde döngü kurulunca **her eleman için** pointer takip edilir, nesnenin tipi belirlenir ve ancak sonra işlem yapılır. Bu **ek yük** (overhead) büyük ve tekrarlı sayısal işlemlerde pahalıdır. NumPy bu boşluğu doldurmak için yaratıldı.

ndarray'in farkı (Slayt 4)

NumPy dizisi (**ndarray**) iki özellik taşır:

- **Homojendir:** tüm elemanlar aynı tiptedir, yani bellekte aynı miktarda yer kaplar.
- **Tek, bitişik bir bellek bloğunda** saklanır.

Bu düzen pointer takibi ve tip kontrolü yükünü kaldırır; daha önemlisi hesaplamaları **C veya Fortran** ile yazılmış, önceden derlenmiş koda devretmeyi sağlar.

```python
import numpy as np
a = np.array([1, 2, 3])
print(a, type(a).__name__, a.dtype)
```

ÇIKTI

```
[1 2 3] ndarray int64
```

Vektörleştirme ve SIMD (Slayt 5–6)

Vektörleştirme, elemanları döngüyle tek tek dolaşmak yerine işlemi **dizinin tamamı** üzerine yazmaktır. Vektörleştirilmiş kod derlenmiş altyapıyı tetikler; bu altyapı **SIMD** (Single Instruction, Multiple Data) ile tek bir komutu birden çok veriye aynı anda uygulayabilir. Ayrıca kod matematik gösterimine benzer, kısa ve okunaklıdır, daha az hata barındırır.

PYTHON

```python
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b, a * 2, a ** 2)
liste = [1, 2, 3]
print([x * 2 for x in liste], liste * 2)   # listede * çoğaltma yapar, çarpma
↪ değil
```

ÇIKTI

```
[11 22 33] [2 4 6] [1 4 9]
[2, 4, 6] [1, 2, 3, 1, 2, 3]
```

Ne kadar hızlı? Ne zaman değil? (Slayt 7–8)

NumPy büyük veride listelerden **kat kat** (orders of magnitude) hızlıdır; bu yüzden Pandas, Scikit-learn ve tüm bilimsel Python ekosistemi NumPy'ı temel alır. **En iyi uygulama:** büyük veri üzerindeki açık döngüleri NumPy ile değiştir. Ancak **tek bir skaler** ya da **çok küçük listeler** için dizi oluşturmanın ek yükü NumPy'ı saf Python'dan yavaş yapabilir.

PYTHON

```python
import time
import numpy as np
liste = list(range(1_000_000))
dizi = np.arange(1_000_000)
t = time.perf_counter(); s1 = [x * 2 for x in liste]; liste_sure =
↪ time.perf_counter() - t
t = time.perf_counter(); s2 = dizi * 2; numpy_sure = time.perf_counter() - t
print('NumPy daha hızlı mı?', numpy_sure < liste_sure)
print('Sonuçlar aynı mı?', s1 == s2.tolist())
```

ÇIKTI

```
NumPy daha hızlı mı? True
Sonuçlar aynı mı? True
```

(Süreler bilgisayara göre değiştiği için burada yalnızca karşılaştırmanın sonucu yazdırıldı; terminalde süreleri kendin görebilirsin.)

N boyutlu dizi, eksenler ve kısıtlamalar (Slayt 9–10)

ndarray, aynı tipte elemanlardan oluşan ve negatif olmayan tam sayı tuple'larıyla indekslenen bir tablodur. Boyutlara **eksen** (axis) denir; 2 boyutlu dizide **axis 0 satırlar, axis 1 sütunlardır**.

ndarray'in üç "kısıtı" (**homojenlik, sabit boyut, dikdörtgen şekil**) aslında performans için bilinçli tercihlerdir: bitişik blok için her elemanın boyutu aynı olmalı ve toplam boyut oluşturulurken sabitlenmelidir. Karışık tipler verilirse NumPy hepsini ortak bir tipe çevirir.

PYTHON
```python
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6]])
print(m)
x = np.array([1, 2.5, 3])
print(x, x.dtype)
```

ÇIKTI
```
[[1 2 3]
 [4 5 6]]
[1.  2.5 3. ] float64
```

ndim, shape, size, dtype, itemsize (Slayt 11–13)

Öznitelik	Anlamı
`ndim`	eksen (boyut) sayısı
`shape`	her boyuttaki uzunluk (tuple); uzunluğu `ndim`'e eşittir
`size`	toplam eleman sayısı = shape elemanlarının çarpımı
`dtype`	eleman tipi (`int32`, `int64`, `float32`, `float64`...)
`itemsize`	eleman başına bayt (float64 → 64 bit = 8 bayt)

PYTHON
```python
import numpy as np
m = np.array([[1.5, 2.0, 3.0], [4.0, 5.0, 6.0]])
print('ndim:', m.ndim, '| shape:', m.shape, '| size:', m.size)
print('dtype:', m.dtype, '| itemsize:', m.itemsize)
print(np.array([1, 2], dtype=np.int32).itemsize)
```

ÇIKTI

```
ndim: 2 | shape: (2, 3) | size: 6
dtype: float64 | itemsize: 8
4
```

Dizi oluşturmak (Slayt 14–18)

Fonksiyon	Ne yapar?
`np.array(liste)`	mevcut veriden; tipi otomatik çıkarır, `dtype=` ile belirtilebilir
`np.zeros(shape)` / `np.ones(shape)`	sıfır / bir dolu (varsayılan **float64**)
`np.empty(shape)`	**başlatılmamış** (içerik rastgele; her elemana sonradan değer atanmalı)
`np.arange(start, stop, step)`	range gibi; **stop dahil değil**
`np.linspace(start, stop, num)`	**iki uç dahil** num eşit aralıklı değer
`np.eye(n)`	n×n birim matris
`np.random.rand(r, c)`	0–1 arası rastgele değerler

Boyut önceden biliniyorsa yer tutucu dizi oluşturmak, diziyi büyütmekten verimlidir. Ondalıklı adımlarda `linspace` tercih edilir: çıktı sayısını kesin kontrol eder ve kayan nokta sorunlarını önler.

PYTHON

```python
import numpy as np
print(np.array([1, 2, 3], dtype=np.float64))
print(np.zeros((2, 3)))
print(np.ones((2, 2)).dtype)
e = np.empty((2, 2))
print(e.shape, e.dtype)            # içerik rastgele olduğu için yazdırmıyoruz
print(np.arange(0, 10, 2), np.arange(5))
print(np.linspace(0, 1, 5))
print(np.eye(3))
np.random.seed(0)
print(np.random.rand(2, 2).round(3))
```

ÇIKTI

```
[1. 2. 3.]
[[0. 0. 0.]
 [0. 0. 0.]]
float64
(2, 2) float64
[0 2 4 6 8] [0 1 2 3 4]
[0.   0.25 0.5  0.75 1.  ]
[[1. 0. 0.]
 [0. 1. 0.]
```

ÇIKTI (DEVAM)

```
 [0. 0. 1.]]
[[0.549 0.715]
 [0.603 0.545]]
```

1 boyutlu indeksleme ve dilimleme (Slayt 19–20)

İndeksleme ve dilimleme veri filtrelemenin ve özellik mühendisliğinin temelidir. 1 boyutlu dizide sözdizimi listelerle aynıdır; indeks 0'dan başlar.

PYTHON

```python
import numpy as np
a = np.arange(10)
print(a[0], a[-1], a[2:5], a[::3])
```

ÇIKTI

```
0 9 [2 3 4] [0 3 6 9]
```

Çok boyutlu indeksleme (Slayt 21–22)

Tek köşeli parantez içinde virgülle: `dizi[satir, sutun]`. Tek başına `:` o eksenin **tamamı** demektir.

PYTHON

```python
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(m[1, 2])
print(m[0, :], m[:, 1])
print(m[:2, 1:])
```

ÇIKTI

```
6
[1 2 3] [2 5 8]
[[2 3]
 [5 6]]
```

reshape ve 3 boyut (Slayt 23–24)

`reshape` elemanları değiştirmeden şekli değiştirir; yeni şeklin toplam eleman sayısı aynı olmalıdır. 3 boyutlu dizide indeks sırası `t[blok, satır, sütun]`'dur.

PYTHON

```python
import numpy as np
print(np.arange(12).reshape(3, 4))
t = np.arange(24).reshape(2, 3, 4)
print(t.shape)
print(t[0, :, :2])
```

PYTHON (DEVAM)

```
print(t[1, 2, 3])
```

ÇIKTI

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 3, 4)
[[0 1]
 [4 5]
 [8 9]]
23
```

View: listede kopya, NumPy'da pencere (Slayt 25–27)

Python listesini dilimlemek elemanların **kopyasını** üretir. NumPy'da basit dilimleme ise bir **view** döndürür: kendi verisi olmayan, orijinalle **aynı veri tamponuna** bakan bir nesne. View değişirse **orijinal de değişir**.

PYTHON

```
import numpy as np
liste = [0, 1, 2, 3, 4, 5]
parca = liste[1:4]
parca[0] = 9
print('liste:', liste)

a = np.arange(6)
v = a[1:4]
v[0] = 9
print('v:', v, '| a:', a)
print('aynı tamponu paylaşıyor mu?', np.shares_memory(a, v))
```

ÇIKTI

```
liste: [0, 1, 2, 3, 4, 5]
v: [9 2 3] | a: [0 9 2 3 4 5]
aynı tamponu paylaşıyor mu? True
```

Bu bir **performans tercihidir**: diziler gigabaytlarca olabilir; her dilim kopya olsaydı bellek kullanımı ve kopyalama yükü performansı çökertirdi. NumPy, listelerin "güvenli ama yavaş" kopyalama davranışı yerine hız ve bellek verimliliğini seçer.

Deep copy: .copy() (Slayt 28–29)

Orijinalden **tamamen bağımsız** bir dizi için `.copy()` açıkça çağrılmalıdır (**deep copy**).

PYTHON

```python
import numpy as np
a = np.arange(6)
c = a[1:4].copy()
c[0] = 9
print('c:', c, '| a:', a, '| paylaşım:', np.shares_memory(a, c))
```

ÇIKTI

```
c: [9 2 3] | a: [0 1 2 3 4 5] | paylaşım: False
```

Kaynaklar (Slayt 29): NumPy User Guide 2.2.0; "NumPy: the absolute basics for beginners" (numpy.org); H. Jani, "NumPy's SIMD-Friendly Design Boosts Performance Over Python Lists"; DATAQUEST, "Stop Using Python Lists and Start Using These Instead".

Sık yapılan hatalar

- `arange`**'in bitişi dahil sanmak:** `np.arange(10, 50)` 50'yi içermez; `np.arange(10, 51)` gerekir.
- **Uyumsuz** `reshape`**:** eleman sayısı tutmazsa `ValueError`.

PYTHON

```python
import numpy as np
np.arange(10).reshape(3, 4)
```

ÇIKTI

```
ValueError: cannot reshape array of size 10 into shape (3,4)
```

- **Var olmayan satıra erişmek** → `IndexError`.
- **Dilimin kopya olduğunu sanmak:** NumPy dilimini değiştirmek orijinali de değiştirir; bağımsız kopya için `.copy()`.
- `np.empty`**'nin sıfır döndürdüğünü sanmak:** içerik rastgeledir.
- `m(1, 2)` **yazmak:** indeksleme köşeli parantezle yapılır: `m[1, 2]`.

Kısa tekrar soruları

1 ndarray'i hızlı yapan iki bellek özelliği nedir?

2 `shape == (2, 3, 4)` olan dizinin `ndim` ve `size` değerleri nedir?

3 float64 dizide `itemsize` kaçtır?

4 `np.linspace(0, 1, 5)` ile `np.arange(0, 1, 0.25)` farkı nedir?

5 `m = np.arange(9).reshape(3, 3)` için `m[:, 0]` ve `m[1:, 1:]` nedir?

6 `a = np.arange(5); b = a[1:3]; b[:] = 0` sonrası `a` nedir? Bunu önlemek için ne yapılmalı?

Cevaplar

1 Homojen olması (her eleman aynı tip ve boyut) ve tek, bitişik bir bellek bloğunda saklanması.

2 `ndim = 3`, `size = 24`.

3 8 bayt.

4 `linspace` iki ucu da dahil eder: `[0. 0.25 0.5 0.75 1.]`; `arange` stop'u dahil etmez: `[0. 0.25 0.5 0.75]`.

5 `[0 3 6]` ve `[[4 5] [7 8]]`.

6 `[0 0 0 3 4]`; `b` bir view'dır. Önlemek için `b = a[1:3].copy()`.

Bölüm 14 · NumPy Bölüm 2

Bölümün amacı

Bu bölüm NumPy dizilerini veri analizine hazırlayan araçları öğretir. Sonunda şunları yapabilmelisin:

- Boolean maskeleme ile filtrelemek, `&`, `|`, `~` ile koşulları birleştirmek, maskeyle değer atamak ve saymak
- `reshape`, `ravel`, `flatten` ve transpoz ile şekil değiştirmek; view/copy farkını gözetmek
- `concatenate`, `vstack`, `hstack` ile birleştirmek ve `split` ile bölmek
- Ufunc'ları ve **broadcasting** kurallarını uygulamak
- Toplama fonksiyonlarını `axis` parametresiyle kullanmak

Konu anlatımı

Boolean indeksleme: maskeleme (Slayt 1–2)

Maskeleme, elemanları bir koşula göre seçmektir ve veri filtreleme/temizlemede temel bir tekniktir. İki adımlıdır:

1. **Maske oluştur:** vektörleştirilmiş bir karşılaştırma, her eleman için True/False içeren bir boolean dizi üretir.
2. **Maskeyi indeks olarak kullan:** True olan konumlardaki elemanlardan oluşan **yeni, 1 boyutlu** bir dizi döner.

PYTHON

```python
import numpy as np
data = np.array([3, 8, 2, 7, 1, 6, 0, 4])
mask = data > 4
print(mask)
print(data[mask])
```

ÇIKTI

```
[False  True False  True False  True False False]
[8 7 6]
```

Bileşik koşullar: &, |, ~ (Slayt 3)

`and`, `or`, `not` dizilerde çalışmaz; bunların yerine `&` (ve), `|` (veya), `~` (değil) kullanılır. Operatör önceliği yüzünden **her koşul parantez içine** alınmalıdır.

PYTHON
```python
import numpy as np
data = np.array([3, 8, 2, 7, 1, 6, 0, 4])
print(data[(data > 3) & (data < 7)])
print(data[(data == 5) | (data < 5)])
print(data[(data == 5) | ~(data < 5)])
```

ÇIKTI
```
[6 4]
[3 2 1 0 4]
[8 7 6]
```

Maskeyle değer atamak ve saymak (Slayt 4)

Maske seçilen elemanlara **değer atamak** için de kullanılır. `np.sum(maske)` True'ları 1 saydığı için koşulu sağlayan eleman **sayısını** verir.

PYTHON
```python
import numpy as np
new_data = np.arange(0, 20)
new_data[new_data % 2 != 0] = -1
print(new_data)
print('Negatif eleman sayısı:', np.sum(new_data < 0))
```

ÇIKTI
```
[ 0 -1  2 -1  4 -1  6 -1  8 -1 10 -1 12 -1 14 -1 16 -1 18 -1]
Negatif eleman sayısı: 10
```

reshape ve -1 (Slayt 5–6)

`reshape` veriyi değiştirmeden yeni bir şekil verir. Yeni şekil **aynı toplam eleman sayısını** içermelidir (değilse `ValueError`). Bellek düzeni izin verirse **view**, vermezse kopya döndürür. Bir boyut için **-1** yazılırsa NumPy onu kendisi hesaplar.

PYTHON
```python
import numpy as np
a = np.arange(12)
print(a.reshape(3, 4))
print(a.reshape(2, -1).shape, a.reshape(-1, 6).shape)
```

ÇIKTI
```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 6) (2, 6)
```

ravel ve flatten (Slayt 7)

İkisi de çok boyutlu diziyi tek boyuta indirir. **ravel** mümkünse **view** döndürür (sonucu değiştirmek orijinali değiştirebilir); **flatten** her zaman **kopya** döndürür.

PYTHON

```python
import numpy as np
m = np.array([[1, 2], [3, 4]])
r = m.ravel()
f = m.flatten()
r[0] = 99
f[1] = 77
print(m)
print(r, f)
```

ÇIKTI

```
[[99  2]
 [ 3  4]]
[99  2  3  4] [ 1 77  3  4]
```

Transpoz (Slayt 8)

Transpoz satırlarla sütunları yer değiştirir: `.T` ya da `np.transpose()`. Yüksek boyutlarda `np.transpose(t, axes=(...))` eksenlerin yeni sırasını alır: (d1, d2, d3) şekli `axes=(2, 0, 1)` ile **(d3, d1, d2)** olur.

PYTHON

```python
import numpy as np
m = np.arange(6).reshape(2, 3)
print(m.T)
t = np.zeros((2, 3, 4))
print(np.transpose(t, axes=(2, 0, 1)).shape)
```

ÇIKTI

```
[[0 3]
 [1 4]
 [2 5]]
(4, 2, 3)
```

Birleştirme (Slayt 9)

- `np.concatenate((a, b), axis=…)`: genel amaçlı; diziler birleştirme ekseni **dışındaki** boyutlarda aynı şekle sahip olmalıdır.
- `np.vstack`: üst üste (satır ekler) = `axis=0`
- `np.hstack`: yan yana (sütun ekler) = `axis=1`

PYTHON
```python
import numpy as np
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
c = np.array([[7], [8]])
print(np.concatenate((a, b), axis=0))
print(np.vstack((a, b)).shape)
print(np.hstack((a, c)))
```

ÇIKTI
```
[[1 2]
 [3 4]
 [5 6]]
(3, 2)
[[1 2 7]
 [3 4 8]]
```

Bölme (Slayt 10)

`np.split(dizi, belirtec, axis)` birleştirmenin tersidir. Belirteç bir tam sayı **N** ise dizi N **eşit** parçaya bölünür (bölünemezse `ValueError`); bir indeks listesi ise bu indeksler **bölme noktalarıdır**.

PYTHON
```python
import numpy as np
x = np.arange(9)
print(np.split(x, 3))
print(np.split(x, [2, 5]))
```

ÇIKTI
```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
[array([0, 1]), array([2, 3, 4]), array([5, 6, 7, 8])]
```

Ufunc'lar (Slayt 11–13)

Ufunc (universal function) dizilerde **eleman eleman** çalışan, döngü gerektirmeyen vektörleştirilmiş fonksiyondur. Aritmetik operatörler ufunc'lara bağlıdır: `a + b` içeride `np.add(a, b)`'yi çağırır.

Kategori	Örnekler	
Aritmetik	`np.add`, `np.subtract`, `np.multiply`, `np.divide`, `np.power`	
Matematiksel	`np.sqrt`, `np.exp`, `np.log`, `np.abs`	

Kategori	Örnekler	
Trigonometrik	`np.sin`, `np.cos`, `np.tan`, `np.arcsin` (radyan)	
Karşılaştırma	`np.greater`, `np.less`, `np.equal` → **boolean dizi**	
Mantıksal/bit	`np.logical_and`, `np.bitwise_or` (`	`), np.bitwise_not (~`)

PYTHON
```
import numpy as np
a = np.array([1, 4, 9])
b = np.array([1, 2, 3])
print(np.add(a, b), a + b, np.multiply(a, b))
print(np.sqrt(a), np.greater(a, 3))
```

ÇIKTI
```
[ 2  6 12] [ 2  6 12] [ 1  8 27]
[1. 2. 3.] [False  True  True]
```

Broadcasting (Slayt 14–16)

Broadcasting, farklı şekillerdeki diziler arasında işlem yapmayı sağlar: küçük dizi büyüğün şekline "esnetilir". Esnetme bellekte **gerçek bir kopya oluşturmaz**, bu yüzden verimlidir. En basit hali dizi ile skaler arasındaki işlemdir.

Şekiller **en sağdaki boyuttan başlayarak** karşılaştırılır:

1. Boyut sayısı farklıysa az boyutlu olanın şekli **soldan 1'lerle** doldurulur: (4,) → (1, 4).
2. Bir boyutta biri **1** ise o dizi diğerinin boyutuna esnetilir.
3. Boyutlar farklı ve **hiçbiri 1 değilse** `ValueError`.

PYTHON
```
import numpy as np
print(np.array([1, 2, 3]) * 10)
matrix = np.ones((3, 4))
vector = np.arange(4)
print(matrix + vector)
print((np.arange(3).reshape(3, 1) * np.arange(4)).shape)
```

ÇIKTI
```
[10 20 30]
[[1. 2. 3. 4.]
 [1. 2. 3. 4.]
 [1. 2. 3. 4.]]
(3, 4)
```

PYTHON
```python
import numpy as np
np.ones((3, 4)) + np.arange(3)
```

ÇIKTI
```
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

Toplama fonksiyonları ve axis (Slayt 17–19)

`np.sum`, `np.mean`, `np.std`, `np.min`/`np.amin`, `np.max`/`np.amax`, `np.prod` varsayılan olarak **tüm dizi** üzerinde çalışır ve **tek bir skaler** döndürür. `axis` parametresi işlemin yönünü belirler:

- `axis=0`: satırlar çökertilir → **her sütun için** bir sonuç
- `axis=1`: sütunlar çökertilir → **her satır için** bir sonuç

PYTHON
```python
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6]])
print(np.sum(m), np.mean(m), np.min(m), np.max(m), np.prod(m), round(np.std(m),
↪  3))
print(m.sum(axis=0), m.sum(axis=1), np.max(m, axis=1))
```

ÇIKTI
```
21 3.5 1 6 720 1.708
[5 7 9] [ 6 15] [3 6]
```

Bölümün kaynakları (Slayt 19): Boolean masking (Medium), NumPy quickstart, NumPy Array Manipulation (Medium), Universal functions (NumPy v2.3), Python Data Science Handbook – Broadcasting, Broadcasting (NumPy v2.3).

Sık yapılan hatalar

- **Maskede `and` / `or` kullanmak** → `ValueError: The truth value of an array ... is ambiguous`.

PYTHON
```python
import numpy as np
data = np.array([3, 8, 2, 7])
data[data > 3 and data < 8]
```

ÇIKTI
```
ValueError: The truth value of an array with more than one element is
↪ ambiguous. Use a.any() or a.all()
```

- **Bileşik koşulda parantezi unutmak:** `data > 3 & data < 7` önce `3 & data` hesaplar.
- **Eşit bölünmeyen diziyi `split` ile N parçaya bölmek** → `ValueError`.
- `vstack`**'te sütun sayıları,** `hstack`**'te satır sayıları farklı diziler** → `ValueError`.
- `ravel` **sonucunun bağımsız olduğunu sanmak:** view olabilir; bağımsız kopya için `flatten` ya da `.copy()`.
- `axis=0`**'ı "satır başına" sanmak:** `axis=0` her **sütun** için sonuç verir.

Kısa tekrar soruları

1 `data = np.array([5, 12, 7, 20])` için `data[(data > 6) & (data < 15)]` nedir?

2 `np.arange(24).reshape(4, -1).shape` nedir?

3 ravel ile flatten arasındaki fark nedir?

4 `np.zeros((2, 3))` ile `np.ones((5, 3))` `vstack` edilirse şekil ne olur? `hstack` edilirse?

5 (2, 3) ile (3,) şekilli diziler toplanabilir mi? (2, 3) ile (2,) peki?

6 `m = np.array([[1, 2], [3, 4]])` için `m.max(axis=0)` ve `m.sum(axis=1)` nedir?

Cevaplar

1 `[12 7]`

2 `(4, 6)`

3 ravel mümkünse view döndürür (orijinal değişebilir); flatten her zaman kopya döndürür.

4 `vstack` → `(7, 3)`; `hstack` → `ValueError` (satır sayıları 2 ≠ 5).

5 (2, 3) + (3,) olur: (3,) → (1, 3) → (2, 3). (2, 3) + (2,) olmaz: sondaki boyutlar 3 ve 2, hiçbiri 1 değil → `ValueError`.

6 `[3 4]` ve `[3 7]`.

Bölüm 15 · Pandas

Bölümün amacı

Bu bölüm tablo biçimindeki veriyle çalışmanın standart aracı **pandas**'ı, tipik bir veri analizi akışını izleyerek öğretir. Sonunda şunları yapabilmelisin:

- **Series** ve **DataFrame** oluşturmak; pandas'ın **etiketlere göre veri hizalama** ilkesini açıklamak
- Veriyi incelemek (`shape`, `dtypes`, `info`, `head`, `describe`), CSV okumak ve yazmak
- `[]`, `loc`, `iloc` ve boolean indeksleme ile veri seçmek ve filtrelemek
- Eksik veriyi bulmak, silmek ve doldurmak
- `concat` ve `merge` ile tabloları birleştirmek; `groupby` ile böl-uygula-birleştir analizi yapmak
- `melt` ve `pivot_table` ile yeniden şekillendirmek; tarihlerle ve zaman serileriyle çalışmak

Konu anlatımı

Pandas nedir, neden pandas? (Slayt 1–5)

Pandas, Python için açık kaynaklı, yüksek performanslı bir veri işleme ve analiz kütüphanesidir; adı ekonometrideki **"panel data"** teriminden gelir. İki temel yapısı vardır: tek boyutlu, etiketli **Series** ve iki boyutlu, etiketli, sütunları farklı tiplerde olabilen **DataFrame**.

- **Elektronik tablolara göre:** büyük veride (milyonlarca satır) yavaşlamaz; işlemler betiklere yazıldığı için analiz **tekrarlanabilir, test edilebilir ve otomatik** olur.
- **SQL'e göre:** SQL veriyi veritabanından çekmek içindir; pandas çekilen veriyle yapılan **analiz ve modelleme** içindir. `GROUP BY`, `JOIN` gibi işlemlerin karşılıkları vardır.
- **R'a göre:** R'ın `data.frame` kolaylığını Python'a getirir.

Kurulum `pip install pandas`; kural olarak `import pandas as pd` ile kullanılır.

PYTHON
```python
import pandas as pd
print(type(pd.__version__).__name__, pd.__version__.split('.')[0])
```

ÇIKTI
```
str 2
```

Series (Slayt 6–8)

Series, tek boyutlu ve **etiketli** bir dizidir; etiketlerine **index** denir. Listeden (index 0, 1, 2...), sözlükten (anahtarlar index olur) ya da `index=` ile oluşturulur.

PYTHON
```python
import pandas as pd
s1 = pd.Series([10, 20, 30])
print(s1)
s2 = pd.Series({'Alice': 85, 'Bob': 92})
print(s2['Bob'])
s3 = pd.Series([1.5, 2.5, 3.5], index=['a', 'b', 'c'], name='olcum')
print(s3.index.tolist(), s3.values, s3.dtype, s3.name, s3.size, s3.shape)
```

ÇIKTI
```
0    10
1    20
2    30
dtype: int64
92
['a', 'b', 'c'] [1.5 2.5 3.5] float64 olcum 3 (3,)
```

DataFrame ve veri hizalama (Slayt 9–12)

DataFrame, aynı index'i paylaşan Series'lerden oluşan bir sözlük gibidir. En yaygın oluşturma yolu **listelerden oluşan sözlüktür**: anahtarlar sütun adları olur ve **tüm listeler aynı uzunlukta** olmalıdır.

PYTHON
```python
import pandas as pd
df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie', 'David'],
                   'Age': [24, 27, 22, 32],
                   'Major': ['CS', 'Math', 'Physics', 'CS']})
print(df)
```

ÇIKTI
```
      Name  Age    Major
0    Alice   24       CS
1      Bob   27     Math
2  Charlie   22  Physics
3    David   32       CS
```

Series'lerden oluşan sözlükte index, tüm index'lerin **birleşimidir**; bir Series'te olmayan etiket **NaN** olur. Bu, pandas'ın temel ilkesi **içsel veri hizalamanın** bir sonucudur: işlemler konuma göre değil **etiketlere göre** eşleşir, eşi olmayan etiket NaN olur. Excel gibi konuma dayalı araçlarda sık görülen yanlış hesaplamaları önler.

PYTHON

```python
import pandas as pd
d = {'one': pd.Series([1.0, 2.0, 3.0], index=['a', 'b', 'c']),
     'two': pd.Series([1.0, 2.0, 3.0, 4.0], index=['a', 'b', 'c', 'd'])}
print(pd.DataFrame(d))
a = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
b = pd.Series([10, 20, 30], index=['b', 'c', 'd'])
print((a + b).tolist())
```

ÇIKTI

```
   one  two
a  1.0  1.0
b  2.0  2.0
c  3.0  3.0
d  NaN  4.0
[nan, 12.0, 23.0, nan]
```

Veriyi incelemek (Slayt 13–17)

- `df.shape` → (satır, sütun) · `df.dtypes` → sütun tipleri (**öznitelik**, parantez yok)
- `df.info()` → özet **yazdırır** (metot): satır sayısı, sütun başına **Non-Null Count** (eksik veri işareti) ve tip. String sütunlar `object` görünür; sayısal olması gereken bir sütunun `object` olması temizlik gerektiğini gösterir.
- `df.head(n)` / `df.tail(n)` → ilk/son n satır (varsayılan 5)
- `df.describe()` → **sayısal** sütunlar için count, mean, std, min, %25, %50 (medyan), %75, max

Merkezi eğilim (mean, medyan, mod) tipik değeri, **yayılım** (std, varyans, aralık) dağınıklığı gösterir. Q1 (%25) ve Q3 (%75) çeyreklerdir; **Q3 − Q1 = IQR**.

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie', 'David'],
                   'Age': [24, 27, 22, 32],
                   'Major': ['CS', 'Math', 'Physics', 'CS']})
print(df.shape)
print(df.dtypes)
df.info()
print(df.describe())
q1, q3 = df['Age'].quantile(0.25), df['Age'].quantile(0.75)
print('IQR:', q3 - q1)
```

ÇIKTI

```
(4, 3)
Name     object
Age       int64
```

ÇIKTI (DEVAM)

```
Major    object
dtype: object
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    4 non-null      object
 1   Age     4 non-null      int64
 2   Major   4 non-null      object
dtypes: int64(1), object(2)
memory usage: 224.0+ bytes
             Age
count   4.000000
mean   26.250000
std     4.349329
min    22.000000
25%    23.500000
50%    25.500000
75%    28.250000
max    32.000000
IQR: 4.75
```

I/O: okuma ve yazma (Slayt 19–23)

Okuyucular `pd.read_*` (csv, json, html, excel, parquet, sql), yazıcılar `df.to_*` biçimindedir. `pd.read_csv` yol, URL ya da `.read()` metodu olan dosya benzeri bir nesne alır. Parametreleri veriyi **okurken temizler**: `sep`, `header`, `index_col`, `usecols` (bellek tasarrufu), `parse_dates`, `na_values`. `pd.read_excel` **openpyxl** gerektirir; `sheet_name=None` tüm sayfaları sözlük olarak okur.

PYTHON

```python
import io
import pandas as pd
metin = '''PassengerId,Survived,Age,Date
1,0,22,2024-01-05
2,1,?,2024-01-06
3,1,26,2024-01-07'''
df = pd.read_csv(io.StringIO(metin), index_col='PassengerId',
                 usecols=['PassengerId', 'Survived', 'Age', 'Date'],
                 na_values=['?'], parse_dates=['Date'])
print(df)
print(df.dtypes)
```

ÇIKTI

```
             Survived   Age       Date
PassengerId
1                   0  22.0 2024-01-05
2                   1   NaN 2024-01-06
3                   1  26.0 2024-01-07
Survived             int64
Age                float64
Date        datetime64[ns]
dtype: object
```

`to_csv` varsayılan olarak index'i de yazar; dosya geri okununca bu, **"Unnamed: 0"** sütunu olarak döner. Bu yüzden CSV'ye yazarken neredeyse her zaman `index=False` kullanılır.

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Name': ['Ada', 'Can'], 'Age': [30, 25]})
df.to_csv('veri.csv')
print(pd.read_csv('veri.csv').columns.tolist())
df.to_csv('veri.csv', index=False)
print(pd.read_csv('veri.csv').columns.tolist())
```

ÇIKTI

```
['Unnamed: 0', 'Name', 'Age']
['Name', 'Age']
```

Seçim: [], loc, iloc (Slayt 25–31)

Etiketler üç amaca hizmet eder: **tanımlama**, **hizalama**, **seçim**. `df['Age']` bir **Series**, `df[['Age', 'Name']]` bir **DataFrame** döndürür. Açık ve güvenli kod için:

	`loc`	`iloc`
Temel	**etiket**	**tam sayı konum**
Dilimin bitişi	**dahil**	**hariç**
Ne zaman?	string/datetime index, etiketle çalışırken	konumla çalışırken

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Age': [24, 27, 22], 'Major': ['CS', 'Math', 'Physics']},
                  index=['Alice', 'Bob', 'Charlie'])
print(type(df['Age']).__name__, type(df[['Age']]).__name__)
print(df.loc['Bob', 'Age'], df.iloc[1, 0])
print(df.loc['Alice':'Bob', 'Age'].tolist())
print(df.iloc[0:2, 0].tolist())
x = pd.DataFrame({'x': [10, 20, 30, 40, 50]})
```

PYTHON (DEVAM)

```
print(len(x.loc[1:3]), len(x.iloc[1:3]))
```

ÇIKTI

```
Series DataFrame
27 27
[24, 27]
[24, 27]
3 2
```

Boolean indeksleme (Slayt 32–34)

Filtreleme, if-then'in **vektörleştirilmiş** karşılığıdır: önce koşuldan bir boolean **maske** oluşur, sonra `df[maske]` ya da `df.loc[maske, sütun]` ile uygulanır. Koşullar `&` (ve), `|` (veya), `~` (değil) ile birleştirilir ve **her biri parantez içine** alınır. Filtrelenen satırlar orijinal index etiketlerini korur.

PYTHON

```
import pandas as pd
df = pd.DataFrame({'Age': [22, 38, 26, 35, 54, 2],
                   'Sex': ['male', 'female', 'female', 'female', 'male',
↪ 'male'],
                   'Pclass': [3, 1, 3, 1, 1, 3]})
mask = df['Age'] > 30
print(mask.tolist())
print(df[mask])
print(df[(df['Age'] > 30) & (df['Sex'] == 'male')].index.tolist())
print(len(df[(df['Pclass'] == 1) | (df['Age'] < 5)]), len(df[~(df['Pclass'] ==
↪ 3)]))
```

ÇIKTI

```
[False, True, False, True, True, False]
   Age     Sex  Pclass
1   38  female       1
3   35  female       1
4   54    male       1
[4]
4 3
```

Eksik veri (Slayt 36–44)

Eksik değerler tipe göre gösterilir: `np.nan` (float), `pd.NaT` (tarih/saat), `None` (object). NaN bir float olduğu için tam sayı sütununa tek bir NaN girince sütun **float64'e yükseltilir**; **nullable** tipler (`'Int64'`, `'boolean'`, `'string'`) bunu `pd.NA` ile önler.

`isna()` (eşi `isnull()`) True/False tablosu verir; `.sum()` ile **sütun başına eksik sayısı** bulunur.

PYTHON
```python
import pandas as pd
print(pd.Series([1, 2, None]).dtype, pd.Series([1, 2, None],
↪ dtype='Int64').dtype)
df = pd.DataFrame({'A': [1, None, 3, None], 'B': [4, None, None, 8], 'C': [7,
↪ None, 9, 10]})
print(df.isna().sum().tolist())
```

ÇIKTI
```
float64 Int64
[2, 2, 1]
```

`dropna()` eksik içeren satırları siler: `axis=1` sütunları, `how='all'` yalnızca tamamı eksik olanları, `thresh=n` en az n geçerli değeri olanları **tutar**. `fillna()` doldurur: sabit değerle, hesaplanmış değerle (ör. ortalama) ya da sıralı olarak (ileri/geri doldurma). Kategorik sütunlar genellikle **mod** (en sık değer) ile doldurulur.

PYTHON
```python
import pandas as pd
df = pd.DataFrame({'A': [1, None, 3, None], 'B': [4, None, None, 8], 'C': [7,
↪ None, 9, 10]})
print(df.dropna().shape, df.dropna(how='all').shape, df.dropna(axis=1,
↪ thresh=3).columns.tolist())
s = pd.Series([1.0, None, None, 4.0])
print(s.fillna(0).tolist(), s.ffill().tolist(), s.bfill().tolist(), s.fillna(
↪ s.mean()).tolist())
e = pd.Series(['S', 'C', None, 'S'])
print(e.fillna(e.mode()[0]).tolist())
```

ÇIKTI
```
(1, 3) (3, 3) ['C']
[1.0, 0.0, 0.0, 4.0] [1.0, 1.0, 1.0, 4.0] [1.0, 4.0, 4.0, 4.0] [1.0, 2.5, 2.5,
↪ 4.0]
['S', 'C', 'S', 'S']
```

Ek bilgi (resmi dokümantasyon): pandas 2.1'den itibaren `fillna(method='ffill')` kullanımdan kaldırılma uyarısı verir; yerine `ffill()` ve `bfill()` metotları önerilir. Kaynak: pandas.pydata.org/docs (pandas.DataFrame.fillna).

Birleştirme: concat ve merge (Slayt 45–50)

- `pd.concat`: aynı yapıdaki tabloları **yığar** (ör. aylık raporlar). Varsayılan `axis=0` alt alta ekler ve orijinal index'leri korur; `ignore_index=True` temiz bir 0, 1, 2… index'i oluşturur. `axis=1` yan yana koyar ve index'e göre hizalar.
- `pd.merge`: aynı varlıklar hakkındaki farklı bilgileri bir **anahtarla** bağlar (SQL JOIN). `on` (ortak sütun), `left_on`/`right_on` (farklı adlı anahtar),

`left_index`/`right_index`. `how`: **inner** (varsayılan, kesişim), **outer** (birleşim), **left**, **right**. `validate='1:1'`, `'1:m'`, `'m:1'` çıktıyı değiştirmez; anahtarların benzersizliğini **doğrular**.

PYTHON

```python
import pandas as pd
jan = pd.DataFrame({'urun': ['A', 'B'], 'satis': [10, 20]})
feb = pd.DataFrame({'urun': ['A', 'C'], 'satis': [15, 5]})
print(pd.concat([jan, feb]).index.tolist(), pd.concat([jan, feb],
↪ ignore_index=True).index.tolist())
df1 = pd.DataFrame({'student_id': ['s1', 's2', 's3'], 'name': ['Ada', 'Can',
↪ 'Ece']})
df2 = pd.DataFrame({'student_id': ['s1', 's2', 's4'], 'course': ['Math',
↪ 'Econ', 'Physics']})
for how in ['inner', 'outer', 'left', 'right']:
    print(how, len(pd.merge(df1, df2, on='student_id', how=how)))
print(pd.merge(df1, df2, on='student_id', how='left'))
```

ÇIKTI

```
[0, 1, 0, 1] [0, 1, 2, 3]
inner 2
outer 4
left 3
right 3
  student_id name course
0         s1  Ada   Math
1         s2  Can   Econ
2         s3  Ece    NaN
```

PYTHON

```python
import pandas as pd
a = pd.DataFrame({'k': [1, 1], 'v': [5, 6]})
b = pd.DataFrame({'k': [1], 'w': [7]})
pd.merge(a, b, on='k', validate='one_to_one')
```

ÇIKTI

```
pandas.errors.MergeError: Merge keys are not unique in left dataset; not a
↪ one-to-one merge
```

groupby: böl, uygula, birleştir (Slayt 51–58)

1. **Böl:** `df.groupby('Sex')` veriyi gruplara ayırır; bu çağrı **henüz hesaplama yapmaz**, tembel bir plan (DataFrameGroupBy) döndürür.
2. **Uygula:** her gruba bir fonksiyon uygulanır: **toplama** (`mean`, `sum`, `count`, `agg`), **dönüştürme** (`transform`: aynı şekil, grup değeri her satıra yayılır), **filtreleme** (`filter`: koşulu sağlamayan grupları tamamen atar).

3 **Birleştir:** toplama sonuçları **grup anahtarlarıyla** indekslenir; dönüştürme ve filtreleme sonuçları **orijinal index'i** korur.

PYTHON
```python
import pandas as pd
df = pd.DataFrame({'Pclass': [3, 1, 3, 1, 3, 3, 1, 3],
                   'Sex': ['male', 'female', 'female', 'female', 'male',
↪ 'male', 'male', 'male'],
                   'Age': [22, 38, 26, 35, 35, None, 54, 2],
                   'Survived': [0, 1, 1, 1, 0, 0, 0, 0]})
print(df.groupby('Sex')['Age'].mean().round(2).to_dict())
print(df.groupby('Pclass')['Survived'].mean().round(2).to_dict())
ozet = df.groupby('Pclass').agg(avg_age=('Age', 'mean'), num_survived=(
↪ 'Survived', 'sum'))
print(ozet)
df['Age'] = df['Age'].fillna(df.groupby('Pclass')['Age'].transform('mean'))
print(df['Age'].round(1).tolist())
print(len(df.groupby('Pclass').filter(lambda g: len(g) >= 4)))
```

ÇIKTI
```
{'female': 33.0, 'male': 28.25}
{1: 0.67, 3: 0.2}
          avg_age  num_survived
Pclass
1       42.333333             2
3       21.250000             1
[22.0, 38.0, 26.0, 35.0, 35.0, 21.2, 54.0, 2.0]
5
```

Yeniden şekillendirme: melt ve pivot_table (Slayt 59–62)

Tidy veride her değişken bir sütun, her gözlem bir satırdır. `pd.melt` sütun başlıkları değer olan (ör. yıllar) **geniş** tabloyu **uzun** formata çevirir (`id_vars`, `var_name`, `value_name`). `pivot_table` tersini yapar: `index` satırlara, `columns` sütunlara, `values` hücrelere gider; aynı hücreye birden çok değer düşerse `aggfunc` uygulanır (varsayılan **ortalama**). pivot_table, groupby + `unstack` birleşimidir.

PYTHON
```python
import pandas as pd
genis = pd.DataFrame({'ulke': ['TR', 'DE'], '2022': [85, 83], '2023': [86,
↪ 84]})
uzun = pd.melt(genis, id_vars=['ulke'], var_name='yil', value_name='nufus')
print(uzun)
df = pd.DataFrame({'Sex': ['m', 'f', 'm', 'f', 'm'], 'Pclass': [1, 1, 3, 3, 3],
                   'Survived': [1, 1, 0, 1, 0], 'Age': [40, 30, 20, 25, 30]})
print(df.pivot_table(values='Age', index='Sex', columns='Pclass'))
print(df.pivot_table(values='Survived', index='Sex', columns='Pclass',
↪ aggfunc='sum'))
```

ÇIKTI

```
  ulke   yil  nufus
0   TR  2022     85
1   DE  2022     83
2   TR  2023     86
3   DE  2023     84
Pclass     1     3
Sex
f       30.0  25.0
m       40.0  25.0
Pclass  1  3
Sex
f       1  1
m       1  0
```

Zaman serileri (Slayt 63–66)

Pandas finansal analiz için geliştirildiğinden tarihlerle çok güçlüdür. Tipler: **Timestamp** (tek an), **DatetimeIndex** (Timestamp'lerden oluşan index), **Timedelta** (süre), **Period** (ay, çeyrek gibi bir aralık). Çoğu tarih string (`object`) olarak yüklenir; `pd.to_datetime` onları Timestamp'e çevirir. Sonra `.dt` ile parçalarına erişilir, tarih aritmetiği yapılır.

PYTHON

```python
import pandas as pd
t = pd.Timestamp('2024-03-15 10:30')
print(t.year, t.month, pd.Timestamp('2024-03-20') - t, pd.Period('2024-03',
↪ freq='M'))
df = pd.DataFrame({'tarih': ['2024-01-05', '2024-02-10', '2024-02-29'],
↪ 'satis': [100, 150, 120]})
print(df['tarih'].dtype, end=' → ')
df['tarih'] = pd.to_datetime(df['tarih'])
print(df['tarih'].dtype)
print(df['tarih'].dt.month.tolist(), df['tarih'].dt.day_name().tolist())
print((df['tarih'].max() - df['tarih'].min()).days)
```

ÇIKTI

```
2024 3 4 days 13:30:00 2024-03
object → datetime64[ns]
[1, 2, 2] ['Friday', 'Saturday', 'Thursday']
55
```

Tarihler index olunca (**DatetimeIndex**) satırlar tarih string'leriyle seçilir (`loc` dilimi bitişi dahil eder). `pd.date_range` düzenli tarihler üretir. `resample` örnekleme sıklığını değiştirir: groupby gibi zamanı aralıklara bölüp her aralığa bir toplama uygular (ör. günlük → haftalık, aylık).

PYTHON

```python
import pandas as pd
idx = pd.date_range('2024-01-01', periods=14, freq='D')
ts = pd.Series(range(1, 15), index=idx)
print(ts.loc['2024-01-03':'2024-01-05'].tolist())
print(ts.resample('W').sum())
print(ts.resample('MS').mean())
```

ÇIKTI

```
[3, 4, 5]
2024-01-07    28
2024-01-14    77
Freq: W-SUN, dtype: int64
2024-01-01    7.5
Freq: MS, dtype: float64
```

Ek bilgi (resmi dokümantasyon): Slayt 64–66'daki örnekler görsel olarak verildiği için bu kısım, slayt başlıklarına ve pandas dokümantasyonundaki `to_datetime`, `date_range` ve `resample` kullanımına göre yazıldı. Kaynak: pandas.pydata.org/docs/user_guide/timeseries.html

Özet ve kaynaklar (Slayt 67–68)

İş akışı: veri alma (`read_csv`) → inceleme (`info`, `describe`) → seçme ve filtreleme (`loc`, `iloc`, maske) → temizleme (`dropna`, `fillna`) → birleştirme (`concat`, `merge`) → analiz (`groupby`, `agg`, `transform`) → yeniden şekillendirme (`melt`, `pivot_table`) → zaman serileri.

Kaynaklar: 10 Minutes to pandas (0.22.0); User Guide, Intro to data structures ve Getting started (pandas 2.3.3), pandas.pydata.org.

Sık yapılan hatalar

- **Series üzerinde `if` kullanmak:** `if df['Age'] > 30:` → `ValueError: The truth value of a Series is ambiguous`. Maske kullan: `df[df['Age'] > 30]`.

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Age': [22, 38, 26]})
if df['Age'] > 30:
    print('yaşlı')
```

ÇIKTI

```
ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(),
↪ a.item(), a.any() or a.all().
```

- **Filtrelemede `and` / `or` yazmak ya da parantezi unutmak:** `&`, `|`, `~` ve her koşul parantez içinde.
- `loc` **ile** `iloc`**'u karıştırmak:** `iloc` etiket kabul etmez; `loc` dilimi bitişi dahil eder.
- `to_csv`**'de** `index=False` **unutmak:** geri okununca fazladan "Unnamed: 0" sütunu oluşur.
- **Farklı uzunlukta listelerle DataFrame kurmak →** `ValueError: All arrays must be of the same length`.
- **Tam sayı sütununda NaN olunca tipin değişmesine şaşırmak:** float64'e yükselir; gerekirse `'Int64'` kullan.
- `groupby`**'ın hemen sonuç ürettiğini sanmak:** bir toplama/dönüştürme/filtre çağrılana kadar yalnızca bir plandır.

Kısa tekrar soruları

1 `pd.Series([1, 2], index=['a', 'b']) + pd.Series([10, 20], index=['b', 'c'])` sonucu nedir?

2 `df.shape` ile `df.info()` arasındaki sözdizimi farkı nedir?

3 `x = pd.DataFrame({'v': [5, 6, 7, 8]})` için `x.loc[1:2, 'v'].tolist()` ve `x.iloc[1:2, 0].tolist()` nedir?

4 `df[(df['Age'] > 30) & (df['Sex'] == 'male')]` neden parantez içerir?

5 `dropna(thresh=2)` ne yapar?

6 inner ve outer merge arasındaki fark nedir? `validate` neyi değiştirir?

7 transform ile agg arasındaki fark nedir?

8 pivot_table'da aggfunc verilmezse ne kullanılır?

9 İki Timestamp'in farkının tipi nedir?

Cevaplar

1 `a` NaN, `b` 12.0, `c` NaN (index'ler hizalanır, birleşimdir).

2 `shape` bir özniteliktir (parantezsiz, tuple döndürür); `info()` bir metottur (parantezle çağrılır, özeti yazdırır).

3 `[6, 7]` (loc'ta bitiş dahil) ve `[6]` (iloc'ta bitiş hariç).

4 `&` karşılaştırma operatörlerinden önce işlendiği için her koşul ayrı ayrı paranteze alınmalıdır.

5 En az 2 eksik olmayan değeri olan satırları tutar, diğerlerini siler.

6 inner yalnızca iki tabloda da olan anahtarları, outer herhangi birinde olan tüm anahtarları tutar. `validate` çıktıyı değiştirmez; anahtar benzersizliği varsayımı bozulursa hata verir.

7 `agg` her grup için tek değer üretir (grup anahtarlarıyla indeksli); `transform` orijinalle aynı şekilde bir sonuç verir, grup değerini her satıra yayar.

8 Ortalama (mean).

9 `Timedelta`.

Bölüm 16 · Ödev 1 – Kavrayış özeti

> ⚠ *Yapay zekâ ile üretildi; slayt ve ödev metniyle karşılaştır.*

> *Bu özet ödevin **çözümü değildir**; ödevin "Analysis" kısımlarında beklenen kavrayışı sınav için toparlar. Kod örnekleri kavramı göstermeye yetecek kadar kısadır. Sürüme veya ortama bağlı her çıktı bu ortamda (CPython 3.9.6, NumPy 2.0.2) çalıştırılarak üretildi; kendi sürümünde farklı çıkabilir.*

Bölümün amacı

Hoca Homework 1'in ara sınava gömüleceğini söyledi: sınav, ödevi yaparken edinmen gereken **kavrayışı** ölçer. Bu özetten sonra şunları açıklayabilmelisin:

- Özyinelemenin bellek modelini (her çağrı bir frame), özyineleme sınırını ve `RecursionError`'ı; döngü ve generator alternatiflerini
- `__add__`, `__radd__`, `__str__`, `__repr__` ve `NotImplemented` mekanizmasını; `sum()`'ın neden `__radd__` istediğini
- CPython, `PyLongObject`, -5…256 küçük tamsayı önbelleği; `is` ile `==` farkı; REPL ile betik farkı
- `int()` ile kesme, `round()` ile banker's rounding, kayan nokta temsil sorunları ve NumPy karşılıkları
- Broadcasting kurallarını, `axis` ile toplamaları, `keepdims`'i ve sütun bazında z-skorunu

Konu anlatımı

Özyineleme ve ikili gösterim (Task 1)

`bin(n)` başında `0b` olan bir string, `format(n, 'b')` öneksiz string döndürür. `int(s, 2)` ikili stringi sayıya geri çevirir.

PYTHON
```
print(bin(5), format(5, 'b'), int('101', 2))
```

ÇIKTI
```
0b101 101 5
```

Özyinelemeli her çağrı, çağrı yığınına (call stack) yeni bir **frame** ekler. Derinliği n olan bir özyineleme O(n) yığın belleği kullanır, döngü ise tek frame'le O(1) ek bellek kullanır. Python kuyruk çağrısı optimizasyonu yapmaz. Derinlik `sys.getrecursionlimit()` değerini aşarsa `RecursionError` fırlatılır:

PYTHON
```python
import sys

def derinlik(n):
    return 0 if n == 0 else 1 + derinlik(n - 1)

print(sys.getrecursionlimit())
try:
    derinlik(5000)
except RecursionError as e:
    print("RecursionError:", e)
```

ÇIKTI
```
1000
RecursionError: maximum recursion depth exceeded in comparison
```

Alternatifler, aynı işi yapmanın önerilen yollarıdır: düz bir `for` döngüsü ya da değerleri **tembel** (lazy) üreten bir generator ifadesi. Generator değerleri istendikçe üretir ve bellekte tüm listeyi tutmaz:

PYTHON
```python
gen = (format(i, 'b') for i in range(4))
print(type(gen).__name__, list(gen))
```

ÇIKTI
```
generator ['0', '1', '10', '11']
```

Okunabilirlik açısından özyineleme, problem doğası gereği özyinelemeli olduğunda (ağaçlar, böl-yönet) daha açıktır. Hız ve bellek açısından ise Python'da döngü genellikle daha iyidir: bu ortamda özyinelemeli sürüm comprehension'dan yaklaşık 2 kat yavaş ölçüldü.

Vector sınıfı ve dunder metotlar (Task 2)

`a + b` ifadesi önce `a.__add__(b)` çağrısını dener. Bu çağrı `NotImplemented` döndürürse Python yansıtılmış metot `b.__radd__(a)`'yı dener. İkisi de çalışmazsa `TypeError` fırlatılır. `NotImplemented` bir hata değil, bir **sinyaldir** (`NotImplementedError` ile karıştırma).

`sum()` toplamaya 0'dan başlar. İlk adım `0 + v1` olduğu için `int.__add__` Vector'ü tanımaz ve `v1.__radd__(0)` gerekir:

PYTHON
```python
class V:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
```

PYTHON (DEVAM)

```
        if isinstance(other, V):
            return V(self.x + other.x, self.y + other.y)
        return NotImplemented

try:
    sum([V(1, 2), V(3, 4)])
except TypeError as e:
    print("TypeError:", e)
```

ÇIKTI

```
TypeError: unsupported operand type(s) for +: 'int' and 'V'
```

`__radd__` 0'ı kabul edince `sum` çalışır. `__str__` kullanıcıya yönelik gösterimdir (`print`, `str`). `__repr__` ise geliştiriciye yöneliktir (REPL, listelerin içi) ve mümkünse nesneyi yeniden üretebilecek biçimde yazılır. `__str__` yoksa `print` `__repr__`'a düşer:

PYTHON

```
class V:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __radd__(self, other):
        return self if other == 0 else NotImplemented
    def __repr__(self):
        return f"V({self.x}, {self.y})"

v = V(1, 2)
print(v, [v], 0 + v)
```

ÇIKTI

```
V(1, 2) [V(1, 2)] V(1, 2)
```

Ödevde tartışılması beklenen **uç durumlar** şunlardır: skalerle toplama (`v + 5` desteklenecek mi, yoksa `NotImplemented` mı dönecek?), farklı boyutlu vektörler (3B ile 2B toplanırsa ne olmalı?) ve `other` nesnesinin Vector olmaması.

CPython ve küçük tamsayı önbelleği (Task 3)

CPython, Python'un C ile yazılmış referans uygulamasıdır (python.org'dan indirilen yorumlayıcı). **PyLongObject**, CPython'da her Python `int` değerini temsil eden C yapısıdır. CPython başlangıçta -5…256 arasındaki tamsayı nesnelerini önceden oluşturur, bu aralıktaki her değer aynı nesneye bağlanır.

`==` **değer** eşitliğini, `is` ise **nesne kimliğini** test eder. Değerler çalışma zamanında üretildiğinde önbelleğin sınırı görünür hale gelir:

PYTHON
```python
import sys
print(sys.version.split()[0])
for v in ["-6", "-5", "256", "257"]:
    a, b = int(v), int(v)
    print(v, a == b, a is b)
```

ÇIKTI
```
3.9.6
-6 True False
-5 True True
256 True True
257 True False
```

Sabit katlama ve kod blokları: aynı kod bloğunda (betik, fonksiyon) geçen aynı sabitler derleyicide tek nesneye birleşebilir. Bu yüzden betikte `a = 257; b = 257` için `a is b` True çıkabilir. REPL'de ise her satır ayrı derlenir ve sonuç False olur. Ödevin sayıları `int("257")` gibi **çalışma zamanında** üretmeyi istemesinin sebebi budur. Aşağıda iki durum `compile()` ile taklit ediliyor:

PYTHON
```python
ns = {}
exec(compile("a = 257\nb = 257", "<betik>", "exec"), ns)
print("aynı blok:", ns["a"] is ns["b"])
exec(compile("a = 257", "<1>", "single"), ns)
exec(compile("b = 257", "<2>", "single"), ns)
print("ayrı satırlar:", ns["a"] is ns["b"])
```

ÇIKTI
```
aynı blok: True
ayrı satırlar: False
```

Önbellek aralığı ve derleyici davranışı sürüme ve uygulamaya bağlıdır (PyPy gibi uygulamalar farklı davranır). Bu yüzden sayılar her zaman `==` ile karşılaştırılmalı, `is` yalnızca `None` gibi tekil nesneler için kullanılmalıdır.

NumPy uygun değildir: NumPy dizileri Python `int` nesneleri değil, ham makine değerleri tutar. Her erişimde yeni bir `np.int64` skaleri oluşturulduğu için `is` CPython önbelleği hakkında bir şey söylemez:

PYTHON
```python
import numpy as np
a = np.array([5, 5])
print(type(a[0]).__name__, a[0] is a[1])
```

ÇIKTI
```
int64 False
```

Kesme ve yuvarlama (Task 4)

`int()` **sıfıra doğru keser**, yani ondalık kısmı atar. `round()` ise en yakın tamsayıya yuvarlar ve tam .5 olduğunda **en yakın çift** sayıyı seçer (round-half-to-even, banker's rounding):

PYTHON
```
print(int(-2.7), int(2.7), round(-2.7))
print(round(0.5), round(1.5), round(2.5), round(-0.5))
```

ÇIKTI
```
-2 2 -3
0 2 2 0
```

Bias (sapma): "yarıyı hep yukarı yuvarla" kuralı çok sayıda .5'li değerde toplamı sistematik olarak büyütür. Çifte yuvarlamada yarılar yarı yarıya aşağı ve yukarı gittiği için ortalama sapma sıfıra yaklaşır. Kesme ise pozitif sayıları hep küçültür ve sapması büyüktür. Para sistemlerinde bu yüzden yuvarlama stratejisi bilinçli seçilir.

Kayan noktalı sayılar ikili sistemde çoğu zaman tam temsil edilemez. `2.675` aslında 2.67499999... olarak saklanır, bu yüzden `round(2.675, 2)` 2.68 değil 2.67 verir. Bu bir hata değildir. Tam ondalık hesap gerekiyorsa `decimal` modülü kullanılır.

PYTHON
```
print(round(2.675, 2), f"{2.675:.20f}")
```

ÇIKTI
```
2.67 2.67499999999999982236
```

Tekrarlanabilir rastgele veri için `random.seed(...)` (NumPy'da `np.random.seed` ya da `np.random.default_rng(seed)`) kullanılır. Aynı seed aynı diziyi üretir.

NumPy karşılıkları şunlardır: `np.trunc` (kesme), `np.rint` ve `np.round` (çifte yuvarlama). Sonuçlar float döner. **Vektörleştirme** döngüyü C'de, Python nesnesi oluşturmadan yapar:

PYTHON
```
import numpy as np
x = np.array([-2.7, -0.5, 0.5, 1.5, 2.5])
print(np.trunc(x), np.rint(x))
```

ÇIKTI

```
[-2. -0.  0.  1.  2.] [-3. -0.  0.  2.  2.]
```

Bu ortamda 10^6 değer üzerinde ölçüldü: `round()`, `int()`'ten yaklaşık 1,1 kat yavaş. Liste üzerinde `int` ile `np.trunc` karşılaştırmasında NumPy yaklaşık 130 kat, liste üzerinde `round` ile `np.rint` karşılaştırmasında yaklaşık 360 kat hızlı. Bu oranlar makineye göre değişir, sıralama ise genellikle aynı kalır.

Broadcasting, toplamalar ve z-skoru (Task 5)

Broadcasting kuralları: iki dizinin şekilleri **sağdan** karşılaştırılır. Eksik boyutlar soldan 1 kabul edilir. Her boyutta değerler ya eşit olmalı ya da biri 1 olmalıdır, aksi halde `ValueError` fırlatılır.

- (3, 4) + (4,) → (4,) önce (1, 4) olur, satırlara yayılır → (3, 4)
- (3, 4) + (3, 1) → sütun vektörü sütunlara yayılır → (3, 4)
- (3, 4) + (3,) → (3,) önce (1, 3) olur, son boyutlar 4 ile 3 uyuşmaz → `ValueError`

PYTHON

```python
import numpy as np
A = np.arange(1, 13).reshape(3, 4)
print((A + np.array([10, 20, 30, 40])).shape, (A + np.array([1, 2, 3]).reshape(
↪ 3, 1)).shape)
print(np.array([1, 2, 3])[:, np.newaxis].shape)
try:
    A + np.array([1, 2, 3])
except ValueError as e:
    print("ValueError:", e)
```

ÇIKTI

```
(3, 4) (3, 4)
(3, 1)
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

axis: `axis=0` satırlar boyunca indirger, yani **her sütun** için bir sonuç verir. `axis=1` **her satır** için bir sonuç verir. (5, 4) bir `X` için `X.max(axis=0)` şekli (4,), `X.mean(axis=1)` şekli (5,) olur, `X.min()` ise tek bir sayıdır.

Sütun bazında z-skoru: `(X - X.mean(axis=0)) / X.std(axis=0)` ifadesinde (4,) şekilli özetler (5, 4)'ün son boyutuyla hizalanır, bu yüzden ek bir işlem gerekmez. **Satır bazında** standardizasyonda ise (5,) şekilli özet (5, 4) ile hizalanmaz. Bu durumda `keepdims=True` ile (5, 1) şekli korunur. Daha yüksek boyutlu veride hangi eksenin indirgendiğini korumak için `keepdims` güvenli yoldur.

PYTHON
```python
import numpy as np
X = np.array([[1., 2.], [3., 6.], [5., 10.]])
Z = (X - X.mean(axis=0)) / X.std(axis=0)
print(Z.mean(axis=0).round(6), Z.std(axis=0))
print(X.mean(axis=1).shape, X.mean(axis=1, keepdims=True).shape)
```

ÇIKTI
```
[0. 0.] [1. 1.]
(3,) (3, 1)
```

`np.std` varsayılan olarak **ddof=0** kullanır, yani popülasyon standart sapmasını (n'e bölme) hesaplar. Örneklem standart sapması için `ddof=1` verilir (pandas'ın `.std()` metodu varsayılan olarak ddof=1 kullanır):

PYTHON
```python
import numpy as np
print(np.std([1, 2, 3, 4]), np.std([1, 2, 3, 4], ddof=1))
```

ÇIKTI
```
1.118033988749895 1.2909944487358056
```

Vektörleştirilmiş işlemler, iç içe Python döngülerine göre çok daha hızlıdır ve daha az hata riski taşır. Makine öğrenmesi ön işleme hatları milyonlarca değer üzerinde çalıştığı için bu fark belirleyicidir.

Sık yapılan hatalar

- `bin(n)` **çıktısını temiz ikili sanmak:** `0b` öneki vardır. Temiz gösterim için `format(n, 'b')` ya da `bin(n)[2:]` kullanılır.
- **Büyük n'de özyineleme:** derinlik sınırı aşılınca `RecursionError` fırlatılır. Sınırı `sys.setrecursionlimit` ile yükseltmek gerçek bir çözüm değildir.
- `__add__` **içinde tip kontrolü yapmamak:** `other.x` erişimi `AttributeError` verir. Doğrusu `NotImplemented` döndürmektir.
- `NotImplemented` **yerine** `NotImplementedError` **döndürmek ya da fırlatmak:** bu durumda yansıtılmış metot hiç denenmez.
- `sum()` **için** `__radd__` **unutmak:** aşağıdaki gibi `TypeError` alınır.

PYTHON
```python
class V:
    def __init__(self, x):
        self.x = x
    def __add__(self, o):
        return V(self.x + o.x)
```

PYTHON (DEVAM)

```
sum([V(1), V(2)])
```

ÇIKTI

```
TypeError: unsupported operand type(s) for +: 'int' and 'V'
```

- **Sayıları `is` ile karşılaştırmak:** sonuç önbelleğe ve sürüme bağlıdır. Sayılar `==` ile karşılaştırılmalıdır.
- **Önbellek sınırını literal sabitlerle test etmek:** sabit katlama sonucu bozar. Sayılar `int("...")` ile çalışma zamanında üretilmelidir.
- `round(2.5)` **sonucunu 3 sanmak:** sonuç 2'dir (çifte yuvarlama). `round(2.675, 2)` de kayan nokta temsili yüzünden 2.67 verir.
- **(3, 4) + (3,) beklemek:** satır başına bir değer eklemek için sütun vektörü gerekir, yani `reshape(3, 1)` ya da `[:, np.newaxis]`:

PYTHON

```
import numpy as np
np.ones((3, 4)) + np.array([1, 2, 3])
```

ÇIKTI

```
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

- **axis yönünü karıştırmak:** sütun ortalaması `axis=0` ile alınır.

Kısa tekrar soruları

1 `format(6, 'b')` ve `bin(6)` ne döndürür?

2 0..n arasını yazan özyinelemeli çözümün yığın belleği nedir, döngülü çözümünki nedir?

3 `__add__` `NotImplemented` döndürürse Python ne yapar?

4 `sum([v1, v2])` neden `__radd__` gerektirir?

5 `a = int("257"); b = int("257")` için `a == b` ve `a is b` ne verir (bu ortamda)?

6 Task 3'te tamsayılar neden çalışma zamanında üretilmelidir?

7 `round(0.5)`, `round(1.5)`, `int(-2.7)` ne verir?

8 Banker's rounding hangi sorunu azaltır?

9 (3, 4) + (3, 1) ve (3, 4) + (3,) işlemlerinin sonucu nedir?

10 `np.std` varsayılan olarak hangi ddof değerini kullanır, `keepdims=True` ne zaman gerekir?

Cevaplar

1 `'110'` ve `'0b110'`.

2 Özyinelemeli çözüm O(n) kullanır (her çağrı bir frame), döngülü çözüm O(1) ek bellek kullanır.

3 Diğer operandın `__radd__` metodunu dener. O da `NotImplemented` dönerse ya da yoksa `TypeError` fırlatılır.

4 `sum` 0'dan başlar ve ilk işlem `0 + v1` olur. `int` Vector'ü bilmediği için `v1.__radd__(0)` çağrılır.

5 `True False` (bu ortamda, CPython 3.9.6, çalıştırıldı). 257 önbellek aralığı -5…256'nın dışındadır.

6 Aynı kod bloğundaki literal sabitler derleyicide tek nesneye birleşebilir (sabit katlama). Bu durumda `is` önbellek dışındaki sayılarda bile True çıkar ve sınır yanlış ölçülür.

7 `0`, `2`, `-2`.

8 Tam .5'li değerlerde hep yukarı yuvarlamanın yarattığı sistematik sapmayı (bias) azaltır.

9 İlki (3, 4) verir. İkincisi `ValueError` verir, çünkü sondan karşılaştırmada 4 ile 3 uyuşmaz.

10 ddof=0 kullanır. `keepdims=True`, satır bazında (ya da daha yüksek boyutta ara bir eksende) indirgenen özetin orijinal diziyle yeniden broadcast edilmesi gerektiğinde, yani eksenin yerinde 1 kalması gerektiğinde kullanılır.